[BUSINESS] FORECASTING AND PREDICTIVE MODELING

LECTURE 7

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 6 ...

- Time series decomposition
- Time series features

Today

- The forecasting workflow

The forecasting workflow

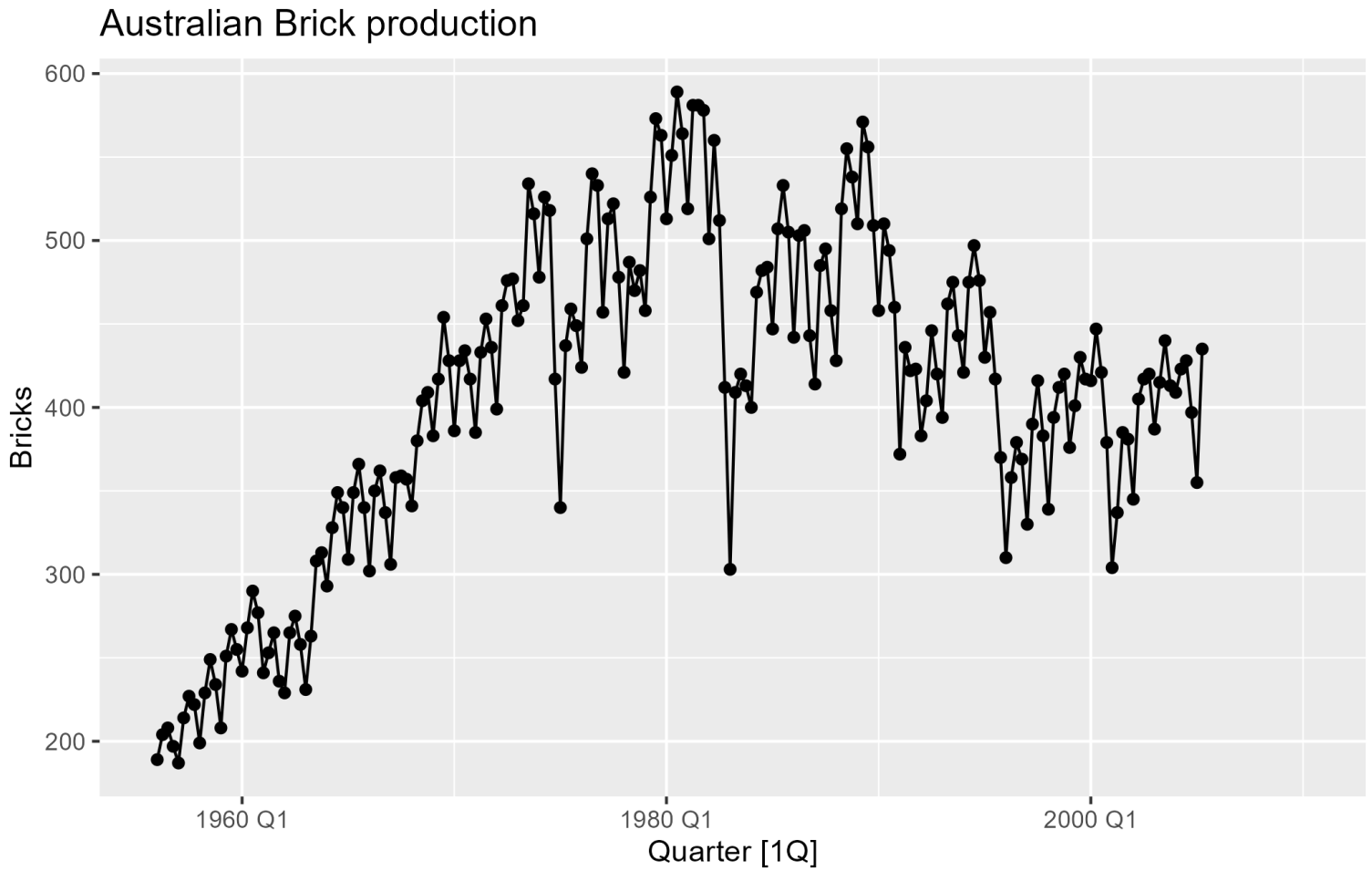
- In general a forecasting workflow is organized in the following way:
  - Data preparation (data cleaning/tidying)
    - Load, filter, transform, identify missing values, outliers, etc.
  - Data visualization
    - Line plot, seasonal plot, ACF, scatter plot, etc.
  - Define a model
  - Train the model
  - Check performance
  - Forecast

Simple forecasting methods

- They are simple methods, not really used for actual forecasting, rather as baseline to evaluate other methods.
- We will talk about 4 simple forecasting methods:
  - mean
  - naive method
  - seasonal naive method
  - drift method
- We will look at them using as example the Australian clay brick production between 1970 and 2004.

```
brick_production <- aus_production |>
  select(Bricks)

brick_production |> autoplot() +
  geom_point() +
  labs(title = "Australian Brick production")
```


Australian Brick production
600
500
400
300
200
Bricks
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]

The mean method I

- In the *mean* method, the forecast for all future values is simply the mean of all historical observations.

$$\hat{y}_{T+h|T} = \frac{y_1 + y_2 + \cdots + y_T}{T}$$

- **Obs:** the notation $\hat{y}_{T+h|T}$ means the forecast/estimated value of y (this is $\hat{y}$) for the time stamp $T + h$, given all the observations $y_1, ..., y_T$.

- In R, it can be computed with the *MEAN* function, passed as argument to the *model* function. Calling the *tidy* function for the model, will give us details about the model parameters.

The mean method II

```
brick_production |>
  model(MEAN(Bricks)) -> brickModel1

tidy(brickModel1)
```

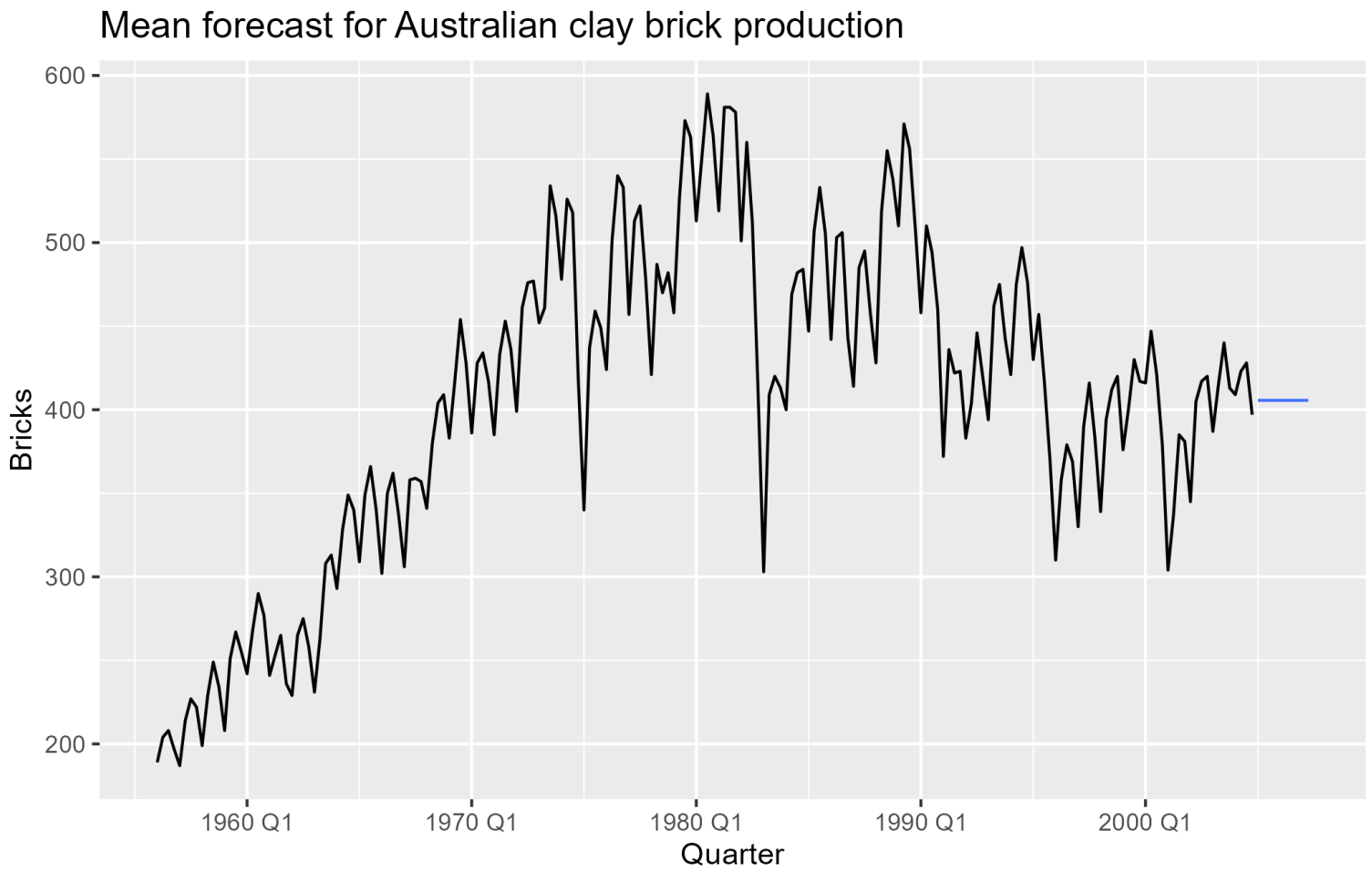
```
# A tibble: 1 × 6
  .model       term  estimate std.error statistic   p.value
  <chr>        <chr>    <dbl>     <dbl>     <dbl>     <dbl>
1 MEAN(Bricks) mean      406.      6.90      58.8 4.88e-126
```

```
brickModel1 |>
  forecast(h = 10) -> brickForecast1

brickForecast1

brickForecast1 |>
  autoplot(level = NULL) +
  autolayer(brick_production, Bricks) +
  labs(title = "Mean forecast for Australian clay brick production")
```

The mean method III



The naive method I

- In the *naive* method, the forecast of all future values is simply the value of the last observation.

$$\hat{y}_{T+h|T} = y_T$$

- In R, it can be computed with the *NAIVE* function, passed as argument to the *model* function.

```
brick_production |>
  model(NAIVE(Bricks)) -> brickModel2

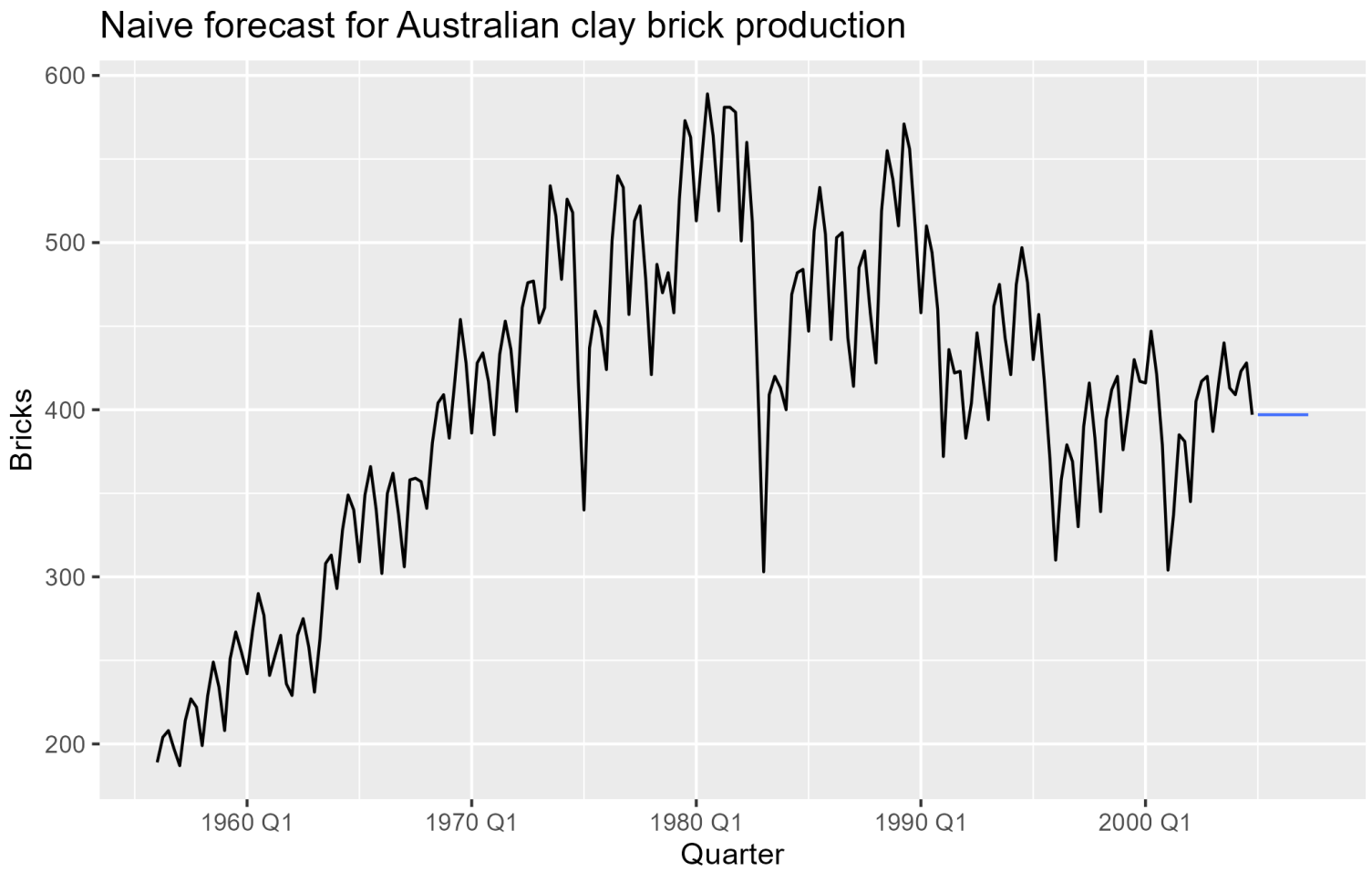
tidy(brickModel2)

brickModel2 |>
  forecast(h = 10) -> brickForecast2

brickForecast2

brickForecast2 |>
  autoplot(level = NULL) +
  autolayer(brick_production, Bricks) +
  labs(title = "Naive forecast for Australian clay brick production")
```

The naive method II



The seasonal naive method I

- This method can be used in case of seasonal data. Like in case of the naive method, we use as forecast the last observation, but from the same season (so we are repeating the last season over and over again).

$$\hat{y}_{T+h|T} = y_{T+h-m*(k+1)}$$

- where m is the seasonal period (ex. 4 in case of quarterly data) and k is the integer part of $\frac{h-1}{m}$ (the number of complete seasons in the forecast period, before $T + h$)
- In R it can be computed with the *SNAIVE* function, which can take as parameter the seasonal lag.

The seasonal naive method II

```
brick_production |>
  model(SNAIVE(Bricks)) -> brickModel3

#brick_production |>
# model(SNAIVE(Bricks) ~ lag("year")) -> brickModel3

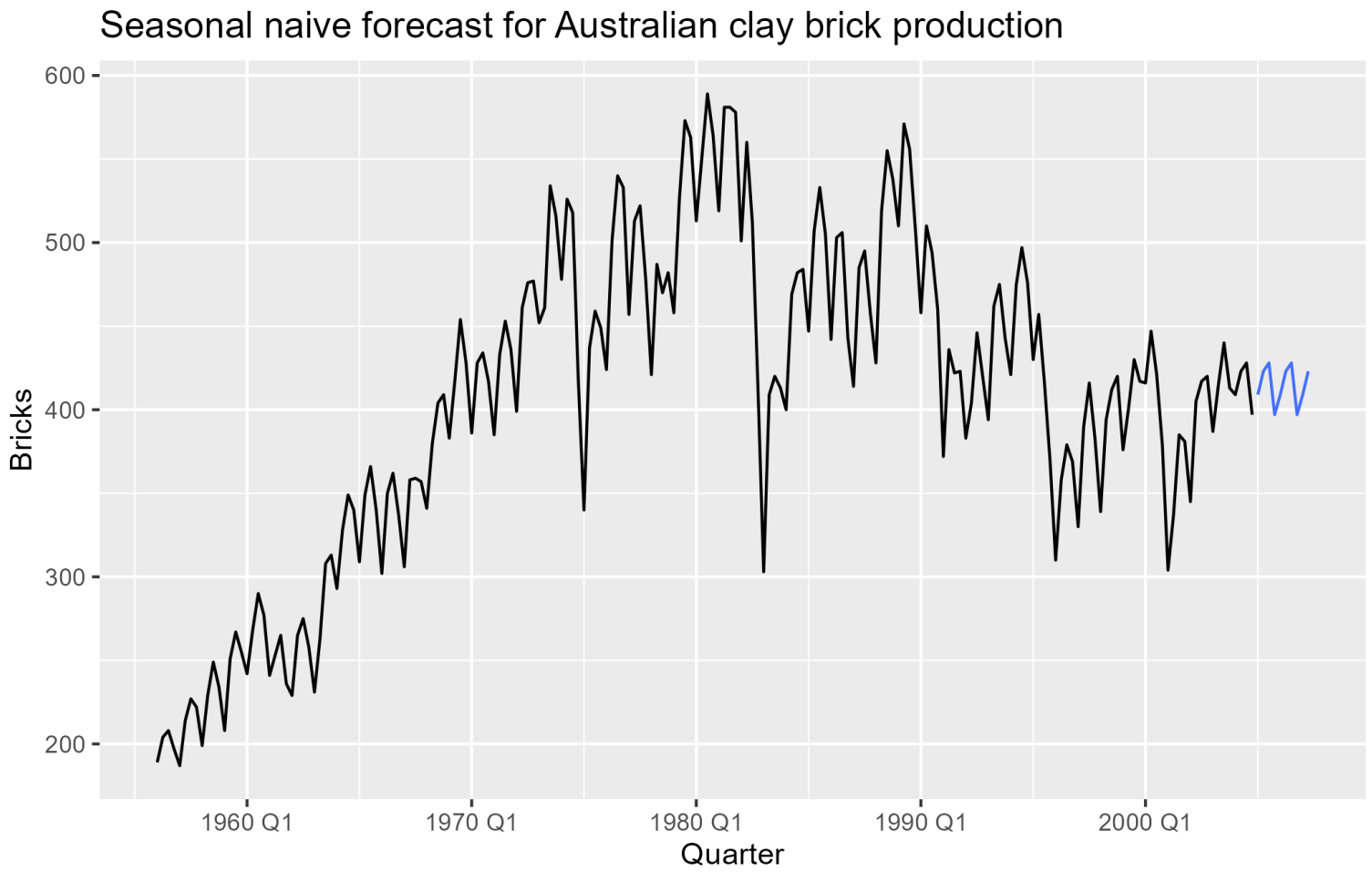
tidy(brickModel3)

brickModel3 |>
  forecast(h = 10) -> brickForecast3

brickForecast3 |> view()

brickForecast3 |>
  autoplot(level = NULL) +
  autolayer(brick_production, Bricks) +
  labs(title = "Seasonal naive forecast for Australian clay brick production")
```

The seasonal naive method III



The drift method I

- Suitable for data which has a trend, in case of the drift model, we allow the predictions to increase / decrease. The amount of change (drift) is the average change seen in the historical data. It is like drawing a line connecting the first and last historical observation and extrapolating from it for forecasting.

$$\hat{y}_{T+h|T} = y_T + h * (\frac{y_T - y_1}{T - 1})$$

- In R it can be computed with the *RW* (random walk) function to which we pass as parameter the *drift* function.

```
brick_production |>
  model(RW(Bricks ~ drift())) -> brickModel4

tidy(brickModel4)
```

The drift method II

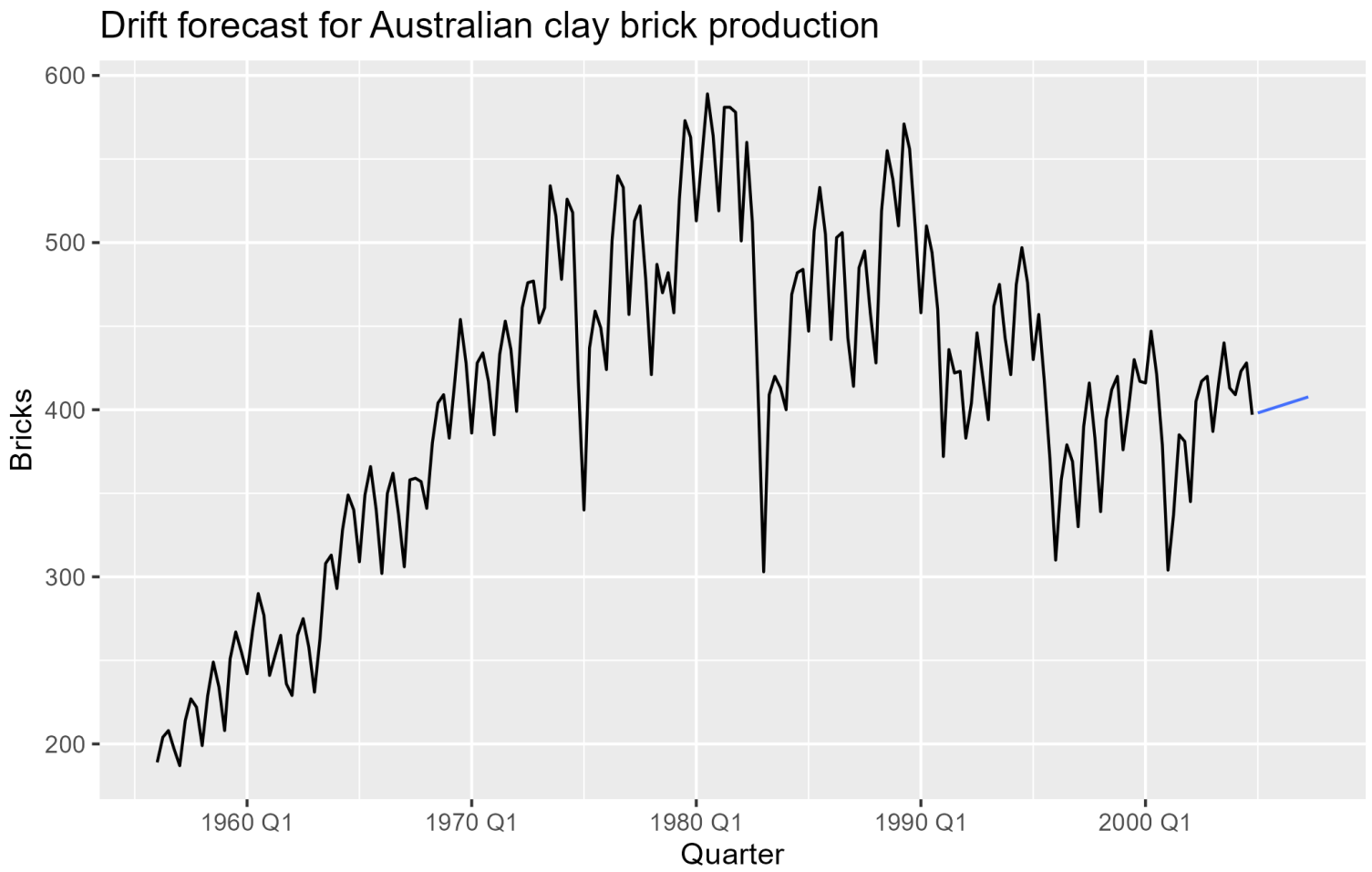
```
# A tibble: 1 × 6
  .model               term  estimate std.error statistic p.value
  <chr>                <chr>    <dbl>     <dbl>     <dbl>   <dbl>
1 RW(Bricks ~ drift()) b         1.07      2.86     0.373   0.710
```

```
brickModel4 |>
  forecast(h = 10) -> brickForecast4

brickForecast4 |> view()

brickForecast4 |>
  autoplot(level = NULL) +
  autolayer(brick_production, Bricks) +
  labs(title = "Drift forecast for Australian clay brick production")
```

The drift method III



Example 1 I

- Let's see all four naive models on the same plot for brick production

```
bricksTrainData <- aus_production |>
  filter(year(Quarter) >= 1992 & year(Quarter) < 2002) |>
  select(Bricks)

bricksAllData <- aus_production |>
  filter(year(Quarter) >= 1992 & !is.na(Bricks)) |>
  select(Bricks)

bricksTrainData |>
  autoplot() +
  geom_point() +
  labs(title = "Australian clay brick production - train data")

bricksAllData |>
  autoplot() +
  geom_point() +
  labs(title = "Australian clay brick production - all data")
```

Example 1 II

```
bricksTrainData |>
  model(mean = MEAN(Bricks),
        naive = NAIVE(Bricks),
        snaive = SNAIVE(Bricks ~ lag("year")),
        drift = RW(Bricks ~ drift())) -> brickModels

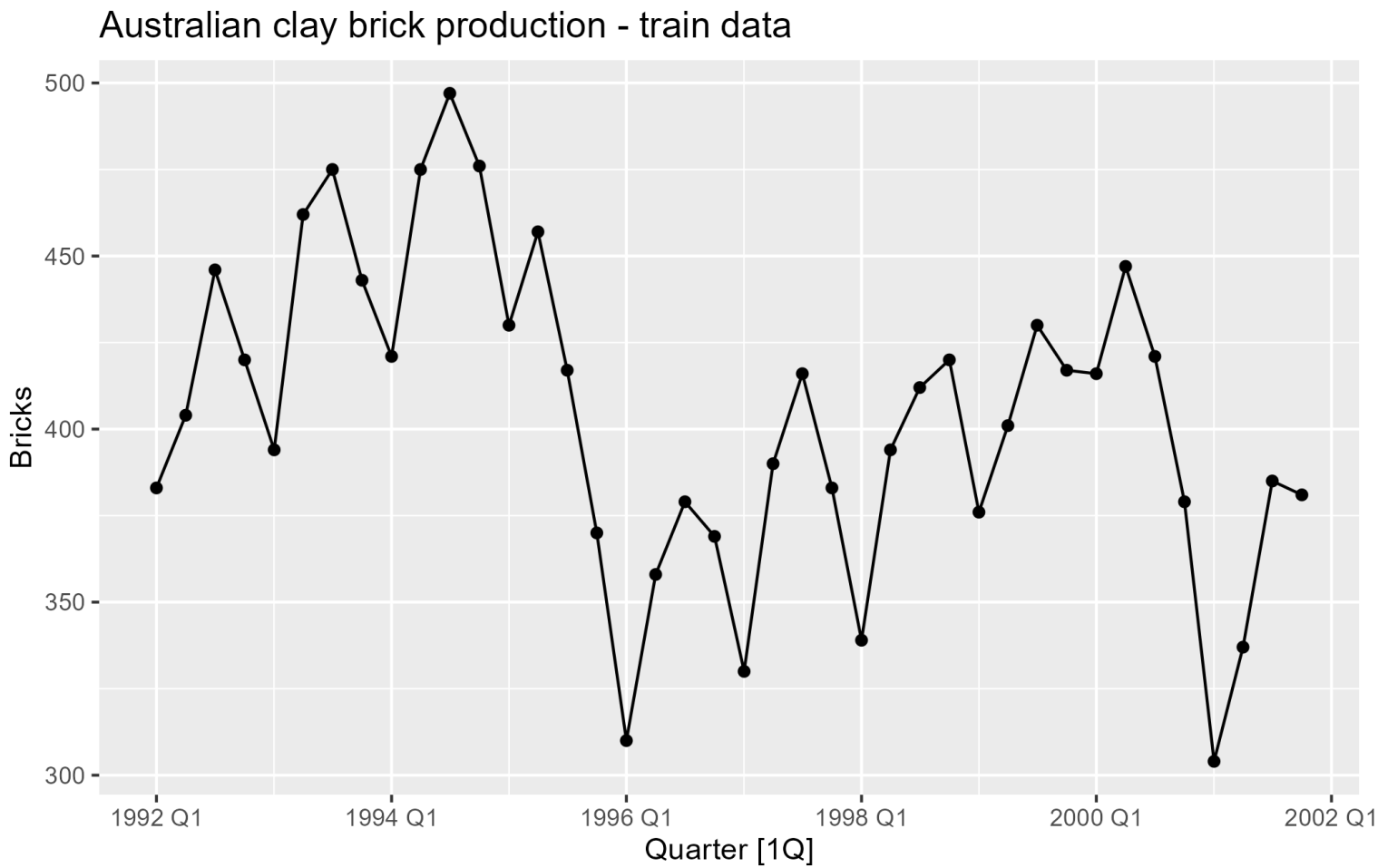
brickModels |> tidy()

brickModels |>
  forecast(h = 14) -> bricksForecast

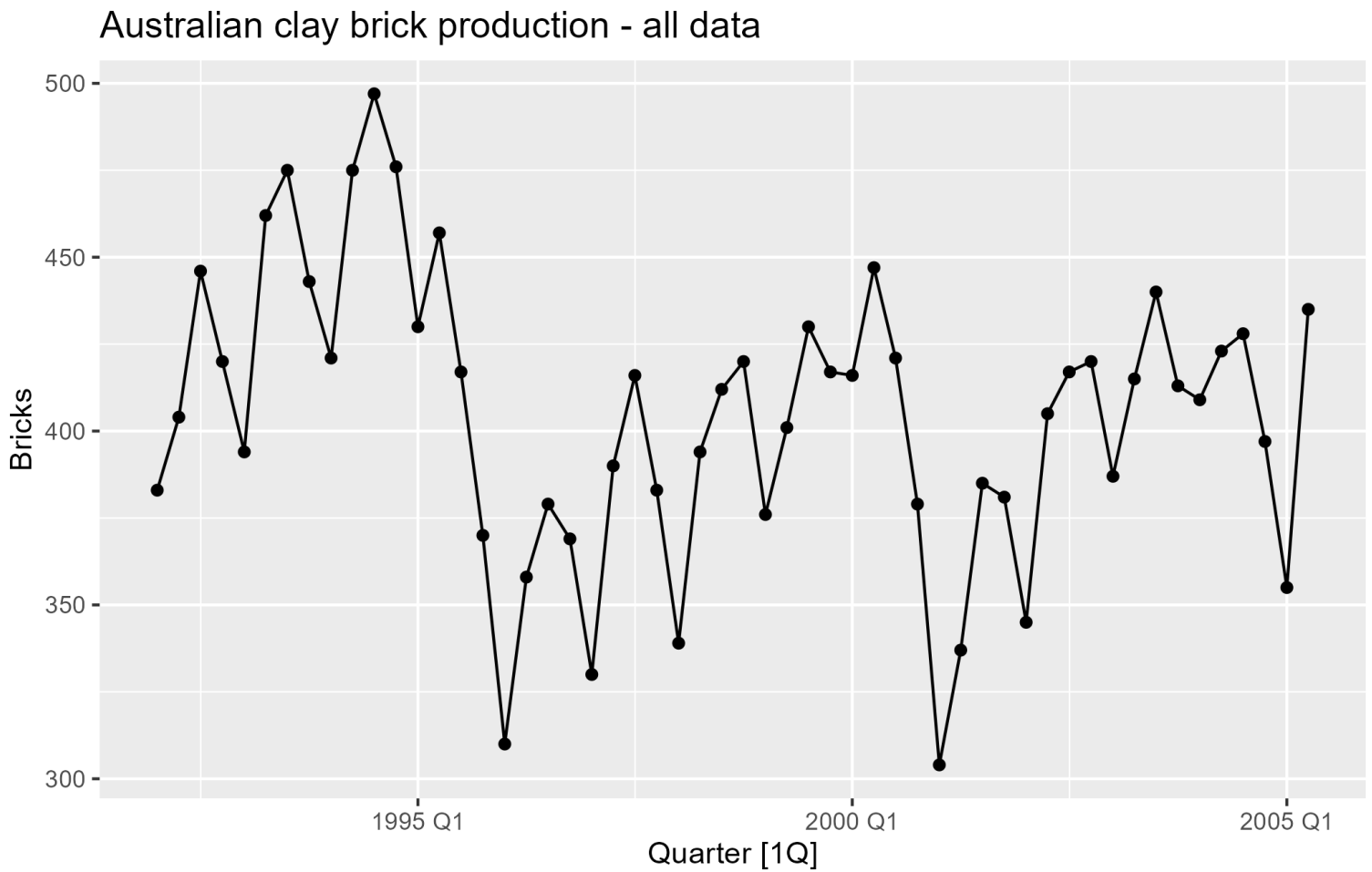
bricksForecast |> view()

bricksForecast |>
  autoplot(bricksTrainData, level = NULL) +
  geom_point(mapping = aes(x = Quarter, y = Bricks)) +
  autolayer(bricksAllData, colour = "black") +
  labs(title = "Forecast of brick production") +
  guides(colour = guide_legend(title = "Forecasts"))
```

Example 1 III



Example 1 IV


Australian clay brick production - all data
Bricks
500
450
400
350
300
1995 Q1
2000 Q1
2005 Q1
Quarter [1Q]

Example 1 V


Forecast of brick production
Bricks
500
450
400
350
300
1995 Q1
2000 Q1
2005 Q1
Quarter
Forecasts
drift
mean
naive
snaive

Example 2 I

- Let's try the same example on the beer production time series from the *aus_production* data set.

```
beerTrainData <- aus_production |>
  filter(year(Quarter) >= 1992 & year(Quarter) < 2007) |>
  select(Beer)

beerAllData <- aus_production |>
  filter(year(Quarter) >= 1992 & !is.na(Bricks)) |>
  select(Beer)

beerTrainData |>
  autoplot() +
  geom_point() +
  labs(title = "Australian beer production - train data")

beerAllData |>
  autoplot() +
  geom_point() +
  labs(title = "Australian beer production - all data")
```

Example 2 II

```
beerTrainData |>
  model(mean = MEAN(Beer),
        naive = NAIVE(Beer),
        snaive = SNAIVE(Beer ~ lag("year")),
        drift = RW(Beer ~ drift())) -> beerModels

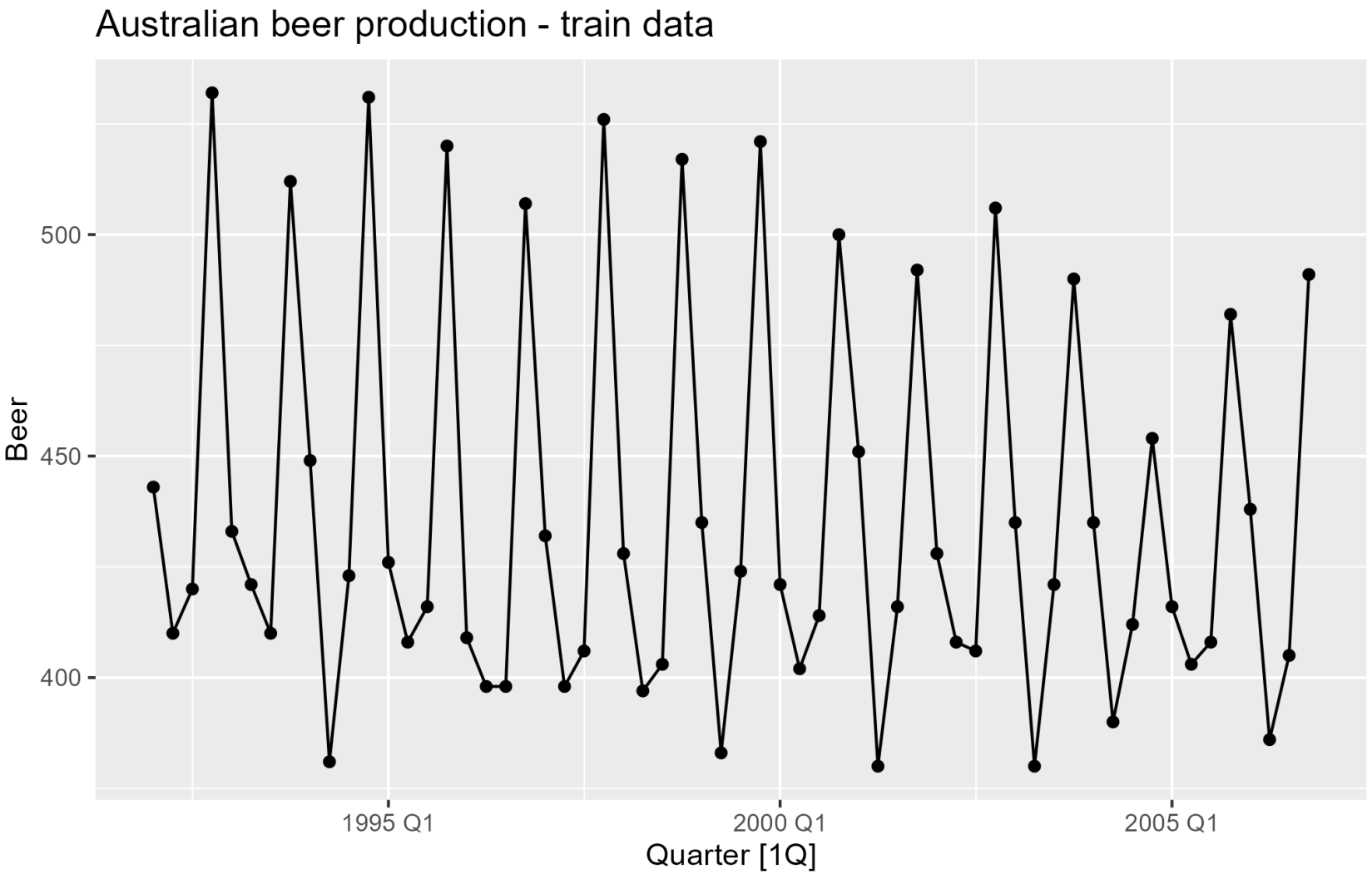
beerModels |> tidy()

beerModels |>
  forecast(h = 14) -> beerForecast

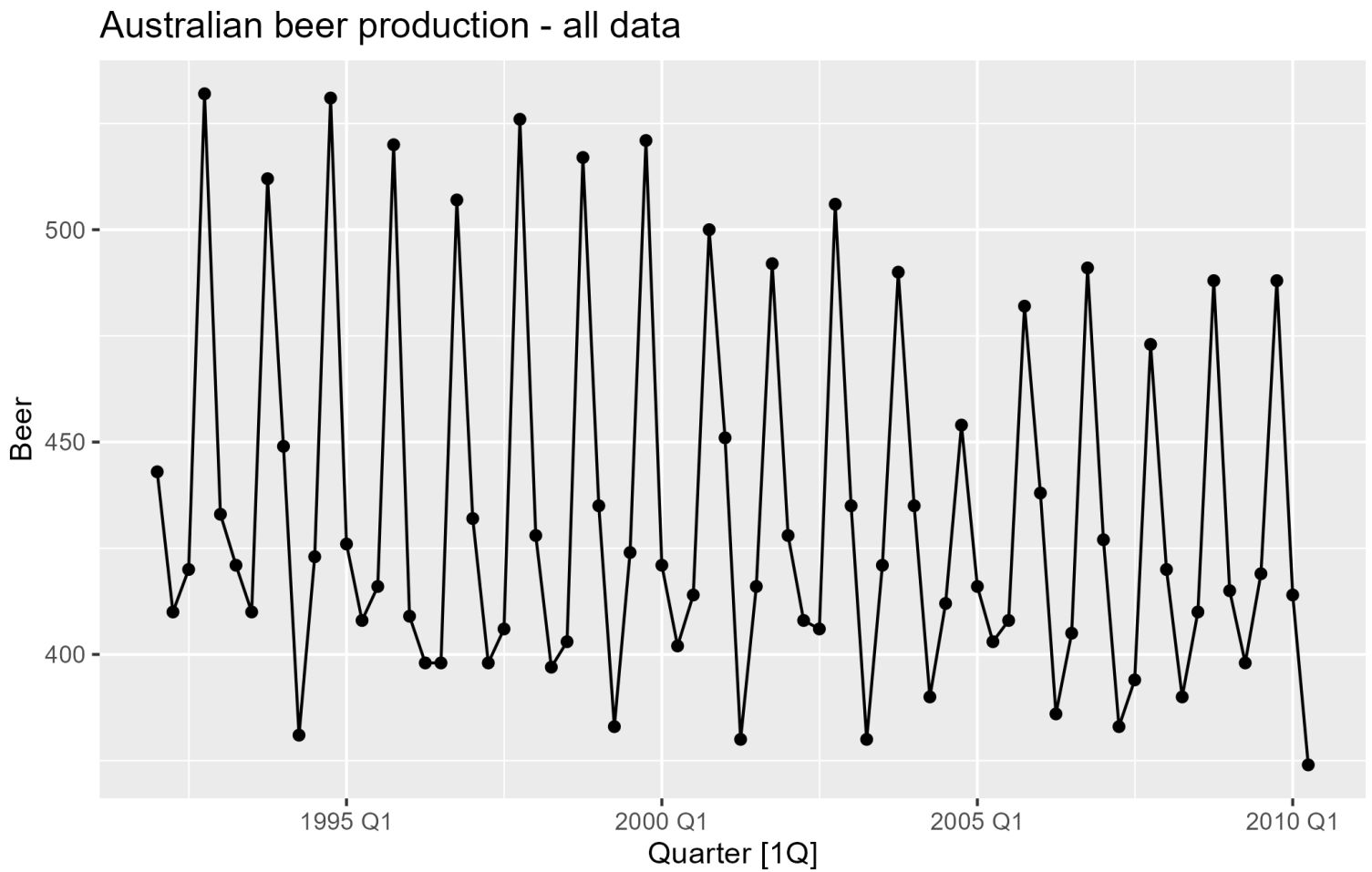
beerForecast |> view()

beerForecast |>
  autoplot(beerTrainData, level = NULL) +
  geom_point(mapping = aes(x = Quarter, y = Beer)) +
  autolayer(beerAllData, colour = "black") +
  labs(title = "Forecast of beer production") +
  guides(colour = guide_legend(title = "Forecasts"))
```

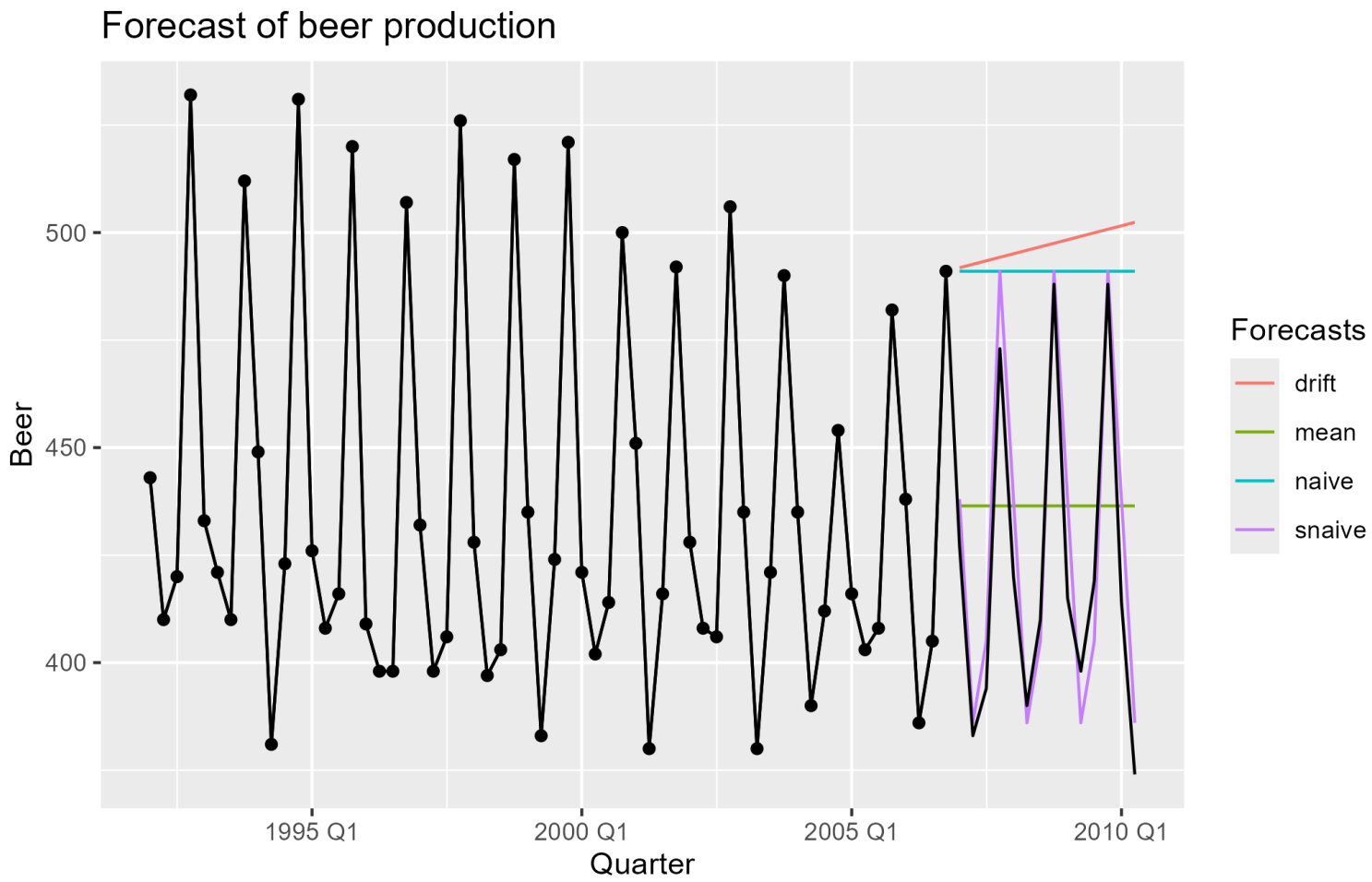
Example 2 III



Example 2 IV


Australian beer production - all data
Beer
500
450
400
1995 Q1
2000 Q1
2005 Q1
2010 Q1
Quarter [1Q]

Example 2 V


Forecast of beer production
Beer
500
450
400
1995 Q1
2000 Q1
2005 Q1
2010 Q1
Quarter
Forecasts
drift
mean
naive
snaive

Example 3

- Let's see a third example using Google stock prices.
- The *gafa_stock* data set contains information about Google, Amazon, Facebook and Apple stock prices between 2014 - 2018.
- We will only use Google data, for *training* the model we will use data from 2015 and we will forecast the first month of 2016.
- Since it is stock prices, the time interval is trading days, which are not consecutive days and are not accepted by some functions. So we will first create a new index for our data (just consecutive numbers), then we will repeat the analysis done in the previous 2 examples.

Example 3 I

```
googleData <- gafa_stock |>
  filter(Symbol == "GOOG" &
           year(Date) > 2014 & yearmonth(Date) <= yearmonth("2016 Jan"))
googleData |> view()

googleData2016DataCount <- googleData |>
  filter(year(Date) == 2016) |> count()
googleData2015DataCount <- googleData |>
  filter(year(Date) == 2015) |> count()

googleData <- googleData |>
  mutate(day= row_number()) |>
  update_tsibble(index = day, regular = TRUE) |>
  select(Close)

googleData

googleTrain <- googleData |>
  head(n = googleData2015DataCount$n[1])
googleTrain |> autoplot() +
  geom_point() +
  labs(title = "Google data used for training")

googleTest <- googleData |>
  tail(n = googleData2016DataCount$n[1])
googleTest |> autoplot() +
  geom_point() +
  labs(title = "Google data used for testing")
```

Example 3 II

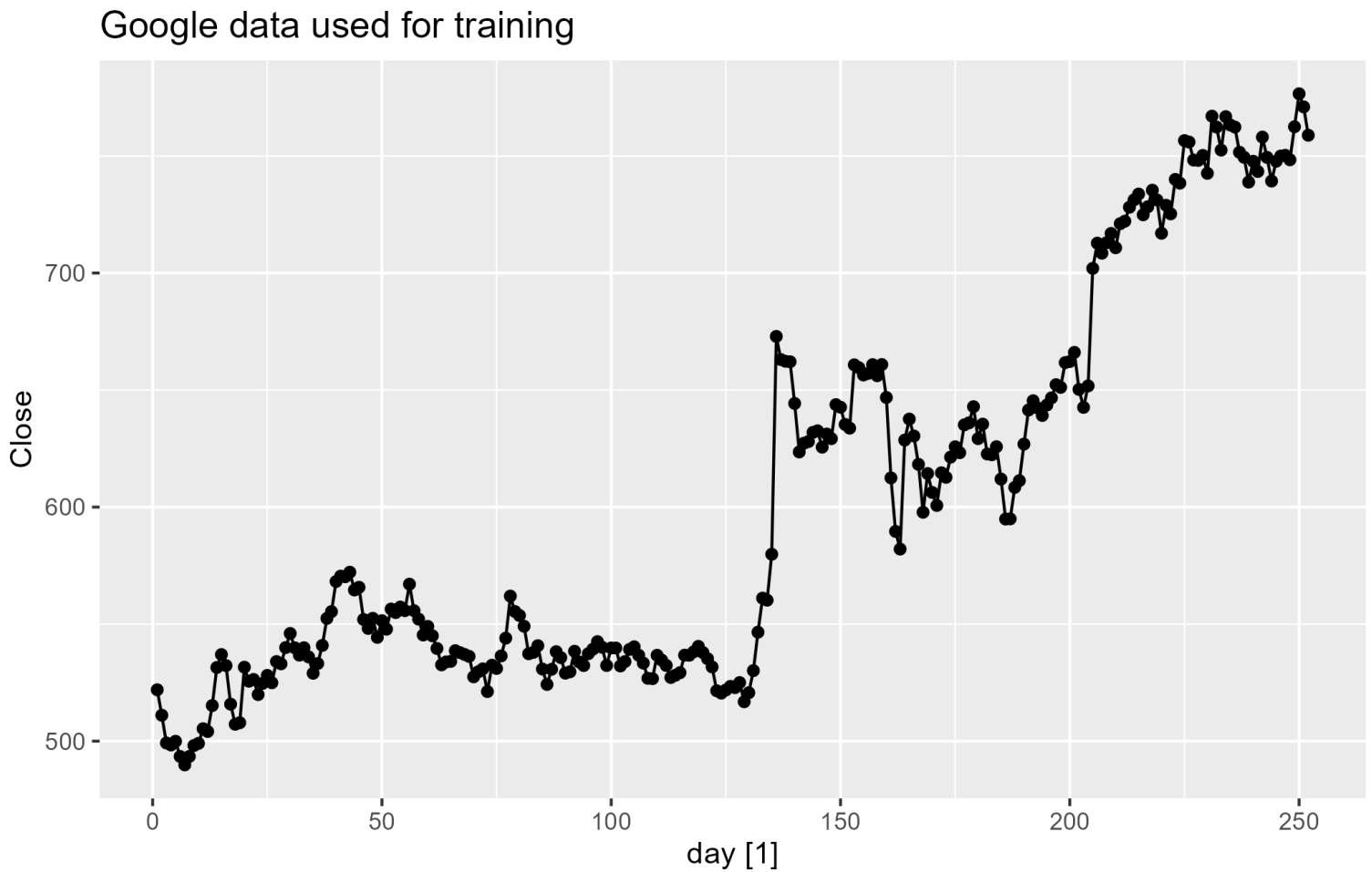
```
googleTrain |>
  model(mean = MEAN(Close),
        naive = NAIVE(Close),
        drift = RW(Close ~ drift())) -> googleModels

googleModels |>
  forecast(h = googleData2016DataCount$n[1]) -> googleForecast

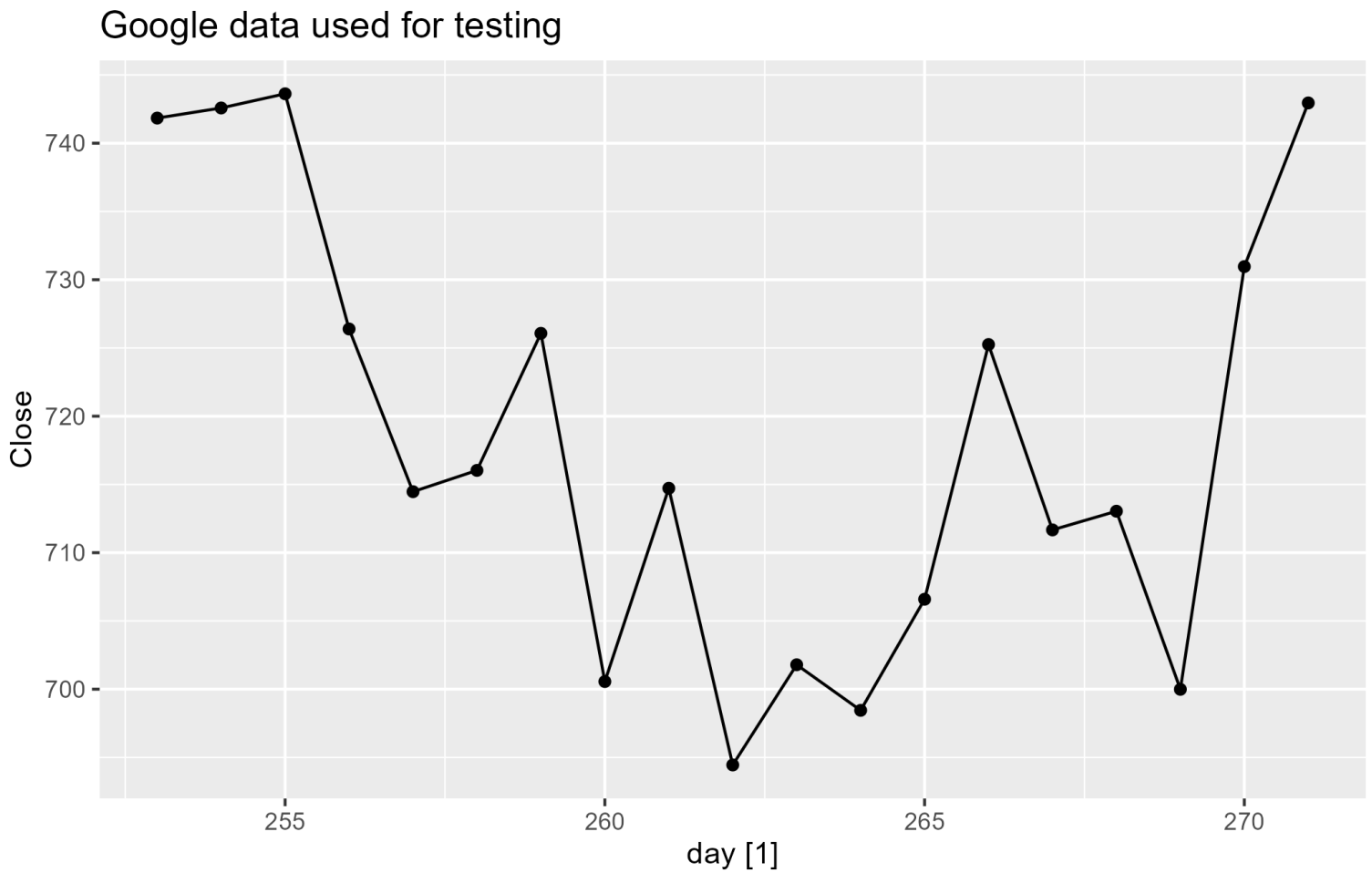
googleForecast |> view()

googleForecast |>
  autoplot(googleTrain, level = NULL) +
  geom_point(mapping = aes(x = day, y = Close)) +
  autolayer(googleData, colour = "black") +
  labs(title = "Forecast of Google stock price") +
  guides(colour = guide_legend(title = "Forecasts"))
```

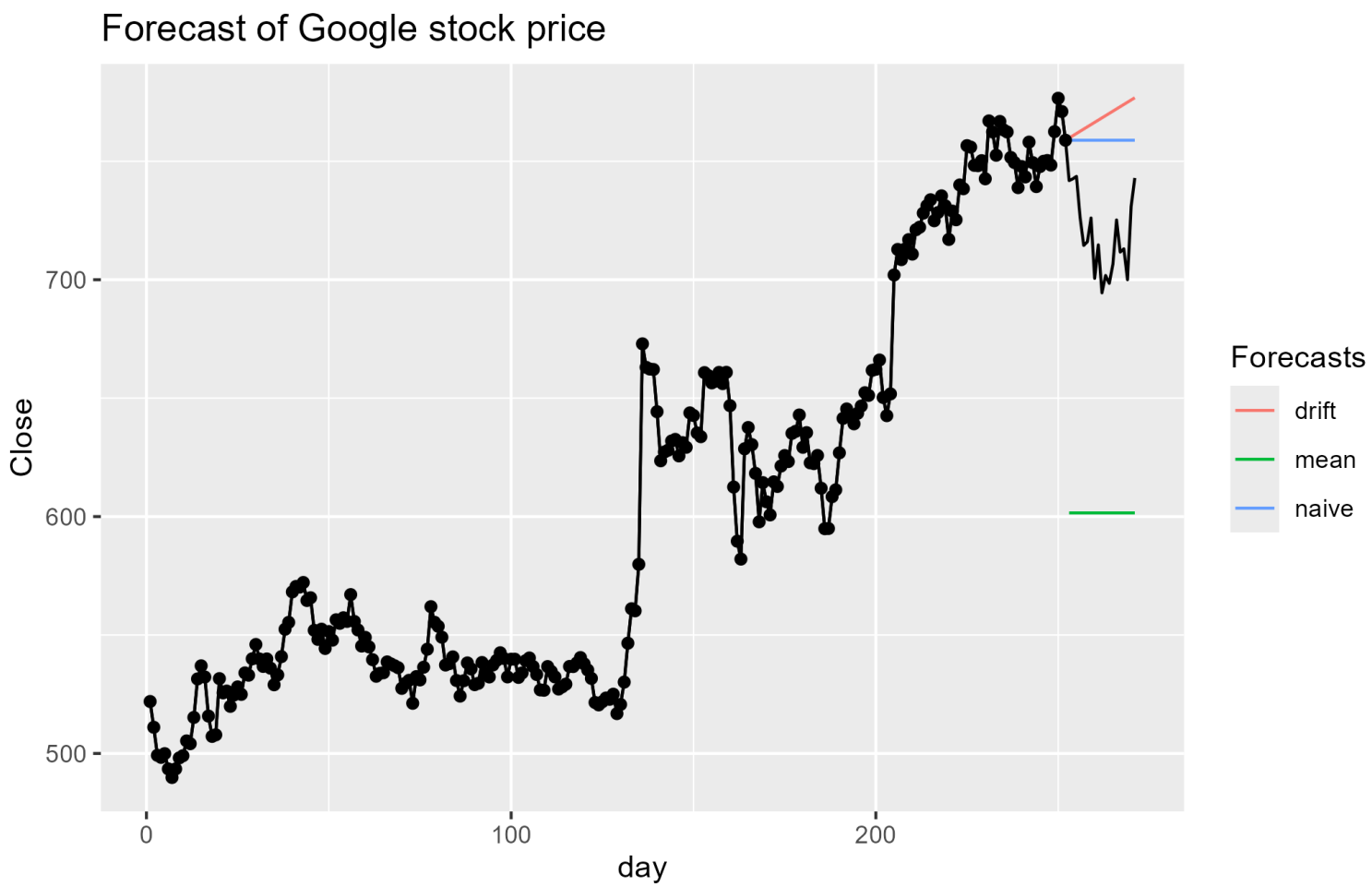
Example 3 III



Example 3 IV


Google data used for testing
740
730
720
710
700
Close
255
260
265
270
day [1]

Example 3 V


Forecast of Google stock price
Close
700
600
500
0
100
200
day
Forecasts
drift
mean
naive

Fitted values

- For each observation from a time series we can do the following: once we have a forecasting model, use it to forecast the value of this observation, using all the previous values. These are called **fitted values** and are denoted by $\hat{y}_{t|t-1}$ or more simply $\hat{y}_t$.
- Are fitted values true forecasts?

Fitted values

- For each observation from a time series we can do the following: once we have a forecasting model, use it to forecast the value of this observation, using all the previous values. These are called **fitted values** and are denoted by $\hat{y}_{t|t-1}$ or more simply $\hat{y}_t$.
- Are fitted values true forecasts?
- It depends on what kind of model produced them. If the model has some parameters, then it is very likely that the parameter of the model was estimated using all available observations, including y_t and even observations for time periods after t. In this case we do not have real forecasts.
- On the other hand, if the model has no parameters, these are real forecasts.

- Which of the discussed baseline models have parameters to be estimated from data?

- Which of the discussed baseline models have parameters to be estimated from data?
- The *naive* and *seasonal naive* baseline models have no parameters (they just repeat the last observation). When we use them, fitted values are real forecasts.
- In case of the *mean* model, we compute the mean of all values, so in this case we do not have real forecasts.
- Similarly, in case of the *drift* model we use all data to estimate the slope of the line between the first and last observation, so we do not have real forecasts.

- **Obs.** This is not the same as time series cross-validation, because for fitted values we build one single model, while in case of time series cross validation we have several. For example, for the mean model:
  - When talking about *fitted values* we compute the mean of all T observations and then this mean is used to compute the fitted values (actually, this mean is the fitted value).
  - When talking about *time series cross-validation* we estimate the mean of the first t observations and use it to forecast observation $t+1$. Then we compute the mean of the first $t+1$ observations and use it to forecast observation $t+2$, etc.

Residuals

- The *residual* of a time series model is the difference between the observation and the fitted value:

$$e_t = y_t - \hat{y}_t$$

- The fitted values and the residuals can be obtained using the *augment* function, passing to it as parameter the fitted models. The function adds three new columns to the data:
  - *.fitted*: the fitted values
  - *.resid*: the residuals
  - *.innov*: the *innovation residuals*
- *Innovation residuals* are important when the perform transformations (for example: $w_t = log(y_t)$) on the data before forecasting. In this case we have the regular residuals $(y_t - \hat{y}_t)$ and the innovation residuals which are computed for the transformed values: $w_t - \hat{w}_t$.

```
beerModels |>
  augment()
```

Residual diagnostics

- You can use residuals to check if the model has captured all the information in the data. A good model will give innovation residuals with the following property:
  - The innovation residuals are uncorrelated.
  - The innovation residuals have zero mean, otherwise they are biased. (Otherwise, if residuals have mean m, simply add this value to all forecasts).
- Any model not respecting these two conditions can be improved. Models respecting these conditions *might* be improved. You should not use the residuals to select a good forecasting method from several methods.

- Optionally, it would be good to have the following two properties as well (useful for computing the prediction interval):
  - The innovation residuals have constant variance (called *homoscedasticity*)
  - The innovation residuals are normally distributed
- Models not respecting these might still be good, but the prediction interval needs to be obtained with different methods. Sometimes using a Box-Cox transformation might help with these properties, but otherwise there is not much you can do to achieve them.

Example I

- Let's look at the residuals for the 2015 Google stock price data set (used previously). For stock market prices, in many cases the naive model is the best (the one where each forecast value is equal to the previous one). Consequently, the residuals are simply the differences between consecutive observations.

Example II

```
google2015Data <- googleTrain
google2015Data |>
  model(naive = NAIVE(Close)) |>
  augment() -> googleNaiveResid

googleNaiveResid

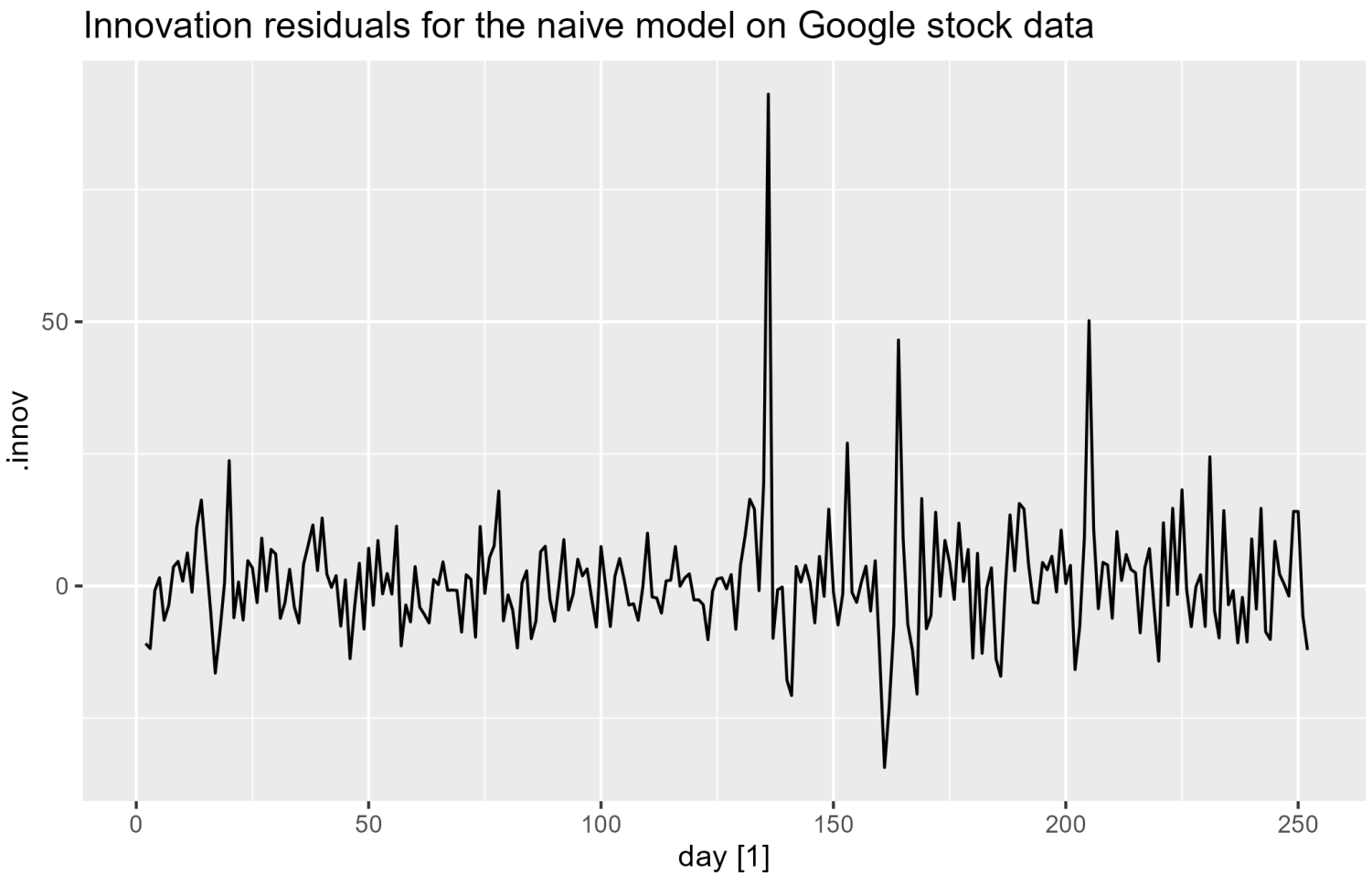
googleNaiveResid |>
  autoplot(.innov) +
  labs("Innovation residuals for the naive model on Google stock data")

googleNaiveResid |>
  filter(is.na(.innov) == FALSE) -> googleNaiveResidNoNA

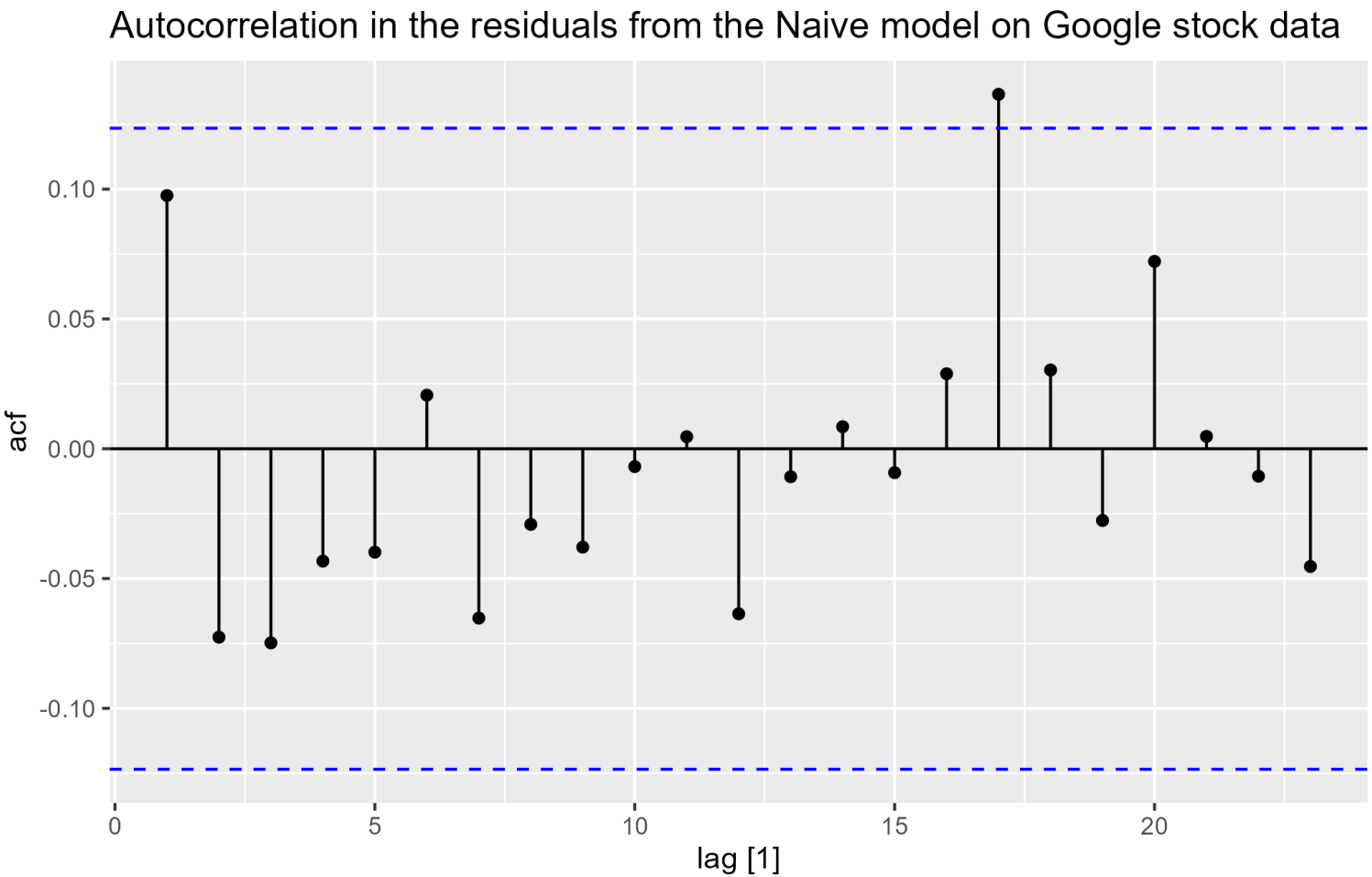
mean(googleNaiveResidNoNA$.innov)
min(googleNaiveResidNoNA$.innov)
max(googleNaiveResidNoNA$.innov)

googleNaiveResid |>
  ACF(.innov) |>
  autoplot() +
  geom_point() +
  labs(title = "Autocorrelation in the residuals from the Naive model on Google
      stock data")
```

Example III


Innovation residuals for the naive model on Google stock data
.innov
50
0
0
50
100
150
200
250
day [1]

Example IV

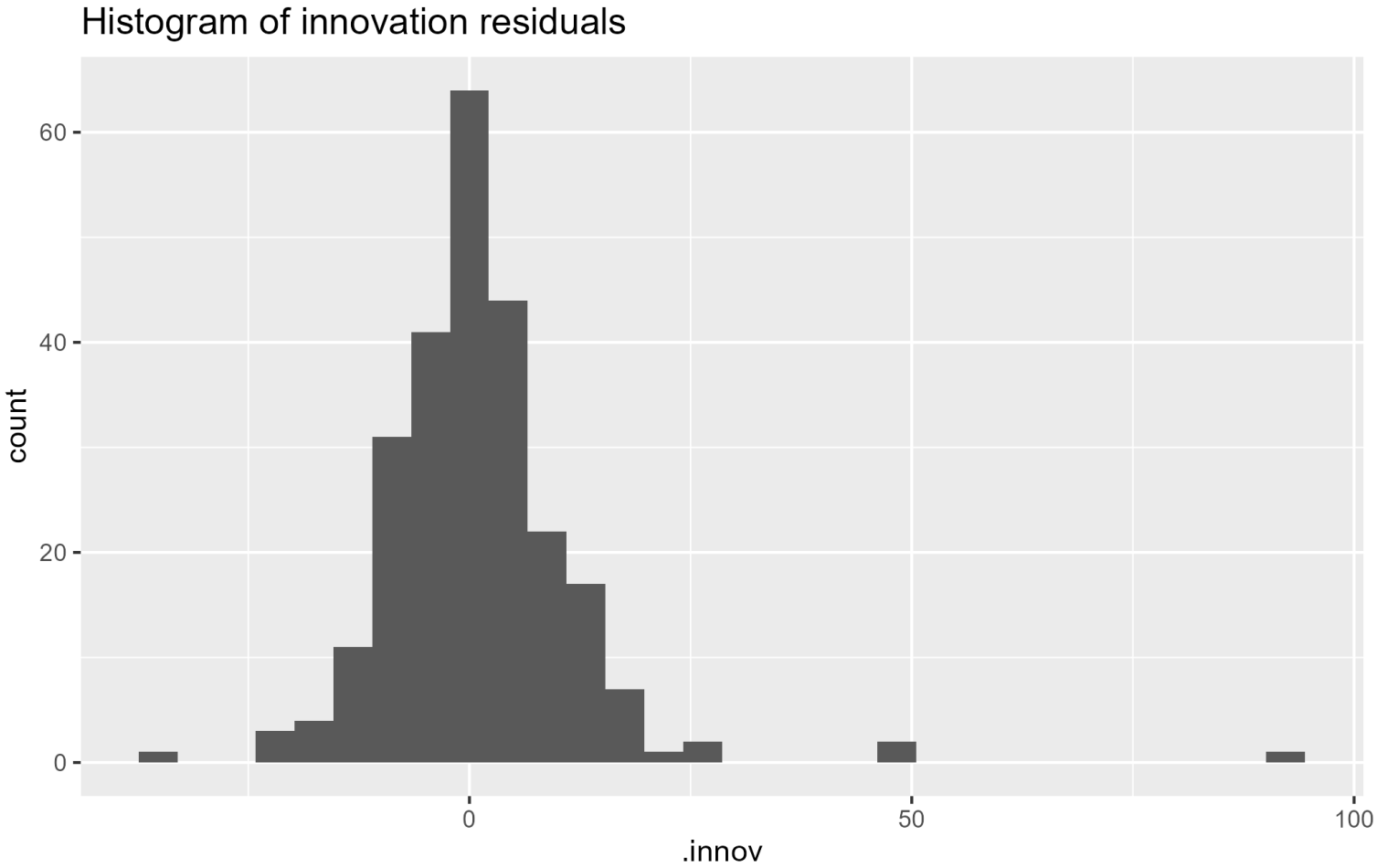

Autocorrelation in the residuals from the Naive model on Google stock data
acf
0.10
0.05
0.00
-0.05
-0.10
0
5
10
15
20
lag [1]

Example

- Since the mean is around 1 (when the values range from -35 to 93) we can say that it is almost zero.
- The ACF plot shows that there is no autocorrelation in data.
- We can conclude that the naive model captures all information from data.

- Let's see the other two properties: normality and constant variance.
- Constance variance can be checked on the autoplot of the innovation residuals. For normality we need a histogram.

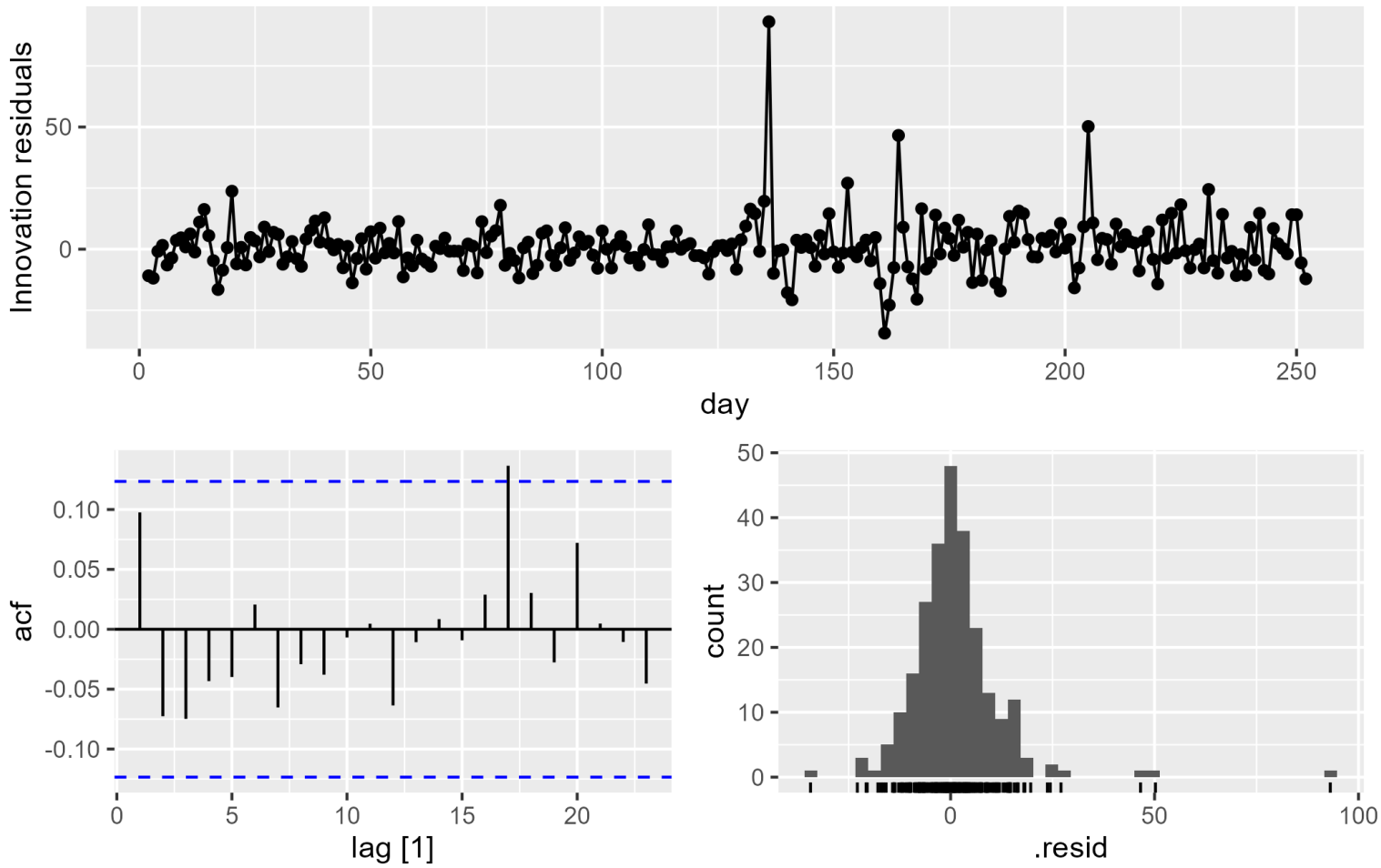
```
googleNaiveResid |>
  ggplot(mapping = aes(x = .innov)) +
           geom_histogram() +
           labs(title= "Histogram of innovation residuals")
```



- We can see that the residuals seem to have a constant variance (with some minor exceptions), but they are not really normally distributed (the right side contains too many values).

- In this example we have used three different plots: a time series plot, an autocorrelation plot and a histogram to check the innovation residuals. We can create all three plots at once with the *gg_tsresiduals* function, which takes as parameter a model (not the augmented one, just the regular model).

```
google2015Data |>
 model(naive = NAIVE(Close)) |>
 gg_tsresiduals()
```


Innovation residuals
50
0
0
50
100
150
200
250
day
acf
0.10
0.05
0.00
-0.05
-0.10
0
5
10
15
20
lag [1]
count
50
40
30
20
10
0
0
50
100
.resid

Autocorrelation tests

- Besides looking at the blue line and determining whether the data is autocorrelated, we can have a more formal test for the autocorrelation of the residuals, which treats all residuals r_k as a group and tests whether the first l autocorrelations are significantly different from what would be expected from white noise.
- There are two related tests: *Box-Pierce* and *Ljung-Box*.

Box-Pierce test

- The *Box-Pierce test* computes the following value:

$$Q = T \sum_{k=1}^{l} r_k^2$$

- where l is the maximum lag considered and T is the number of observations. If Q is small, that means that we have no autocorrelation in the data.
- For non-seasonal data the suggested value for l is 10, while for seasonal data it is suggested to use $2 * m$, where m is the period of the season.
- However, l should not exceed the value of $\frac{T}{5}$, so if the previous values are larger than this limit, $\frac{T}{5}$ should be used.

Ljung-Box text

- Another test to see if there is autocorrelation in the data, which is more accurate than the *Box-Pierce* test is the *Ljung-Box* test, which computes the following value:

$$Q^* = T * (T+2) \sum_{k=1}^{l} (T-k)^{-1} * r_k^2$$

- where the same notations are used as for the Box-Pierce test.
- Large values of Q* suggest that the autocorrelation does not come from white noise.

- What does *large* Q and Q* value mean? You need to look at the p-value. The *null hypothesis* of these tests is that the data is significantly different from the white noise. If the *p-value* returned by the test is high (ex. greater than 0.05) we can *reject* the null hypothesis and conclude that the data is not significantly different from white noise.

```
googleNaiveResid |>
  features(.innov, box_pierce, lag = 10)
```

```
.model      bp_stat bp_pvalue
  <chr>       <dbl>     <dbl>
1 naive       7.74      0.654
```

```
googleNaiveResidl |>
    features(.innov, ljung_box, lag = 10)
```

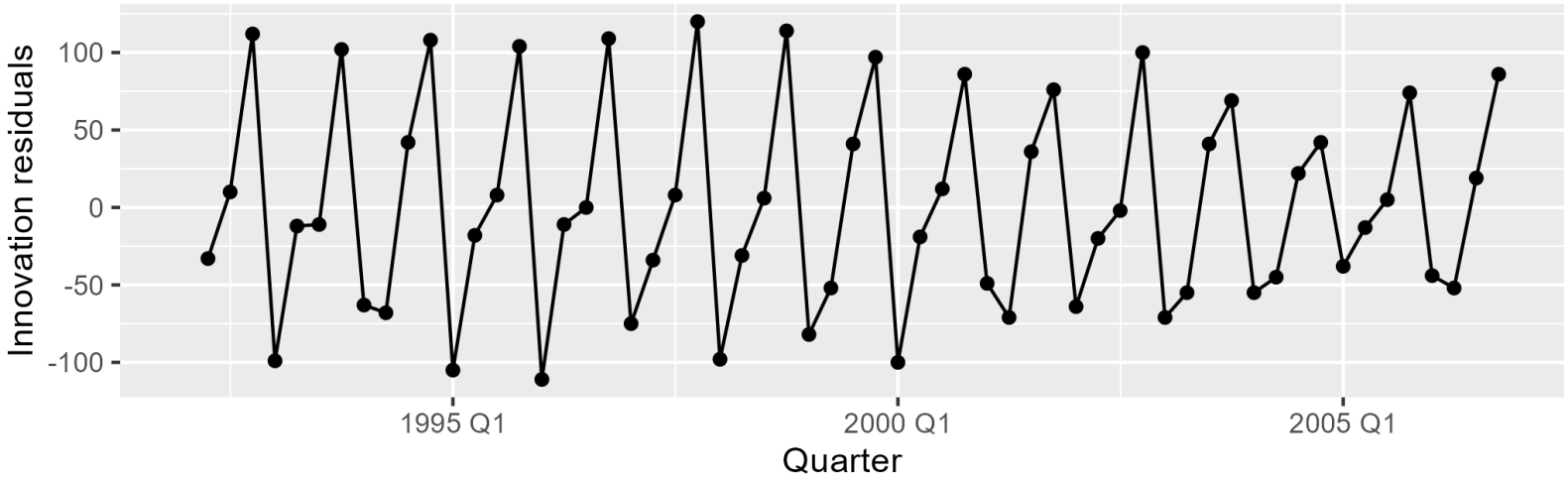
```
  .model lb_stat  lb_pvalue
  <chr>    <dbl>     <dbl>
1 naive     7.91     0.637
```

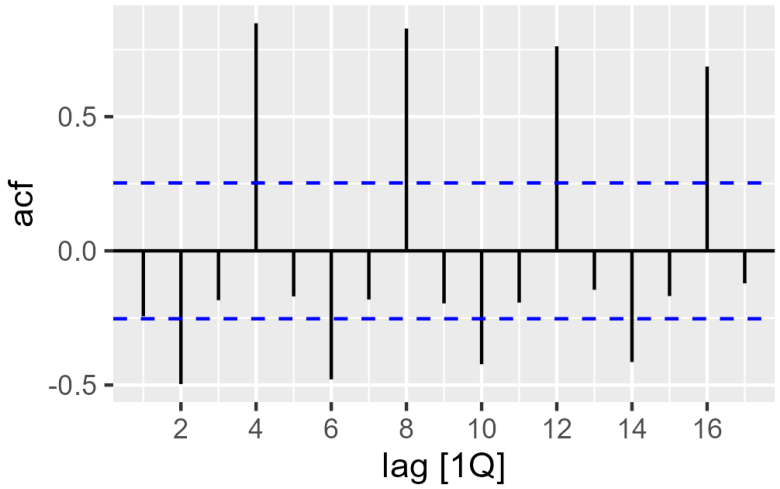
- Let's look at these two tests for another model as well

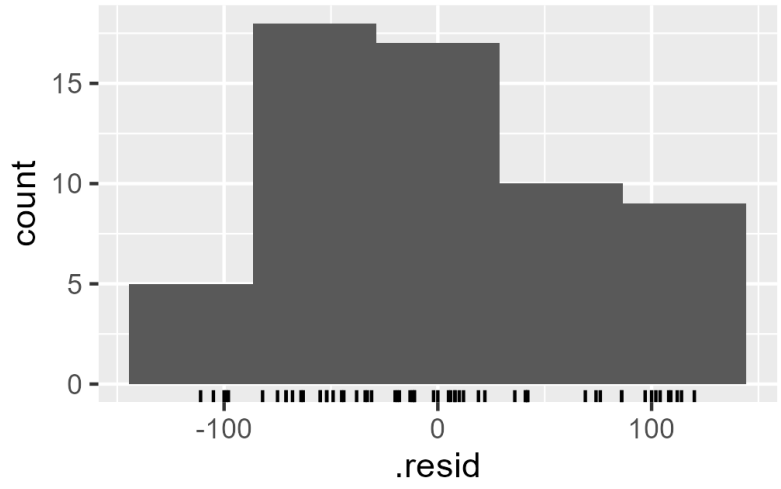
```
beerTrainData |>
  model(naive = NAIVE(Beer)) |>
  gg_tsresiduals()

beerTrainData |>
  model(naive = NAIVE(Beer)) |>
  augment() |>
  features(.innov, box_pierce, lag = 8)

beerTrainData |>
  model(naive = NAIVE(Beer)) |>
  augment() |>
  features(.innov, ljung_box, lag = 8)
```








```
  .model bp_stat bp_pvalue
  <chr>    <dbl>     <dbl>
1 naive     111.         0
```

```
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 naive     127.         0
```

- Both tests agree that the residuals from the Naive model on the Google stock price data come from white noise, while those for the beer production data are not white noise (this is obvious if you look at the ACF plot).

Prediction intervals

- While we have looked at (and plotted) only the forecast values, forecasting methods actually return a probability distribution of possible forecast values. Out of these values, the point forecast is the mean of the distribution. (This is why we used the $level = NULL$ parameter when plotting the baseline methods, to plot only the point forecast).
- For most models this probability distribution is the normal distribution.
- In forecasting prediction intervals are important, they show us how much uncertainty we have in our forecast. This is why, having just the point forecast is basically useless.

- A *prediction interval* is an interval in which we expect the actual value y_t to be with a certain probability p. For example, assuming a normal distribution for the future values, the 95% prediction interval for an h-step forecast is:

$$\hat{y}_{T+h|T} = \pm 1.96 * \hat{\sigma}_h$$

- where $\hat{\sigma}_h$ is the standard deviation of the h-step forecast distribution.
- The value 1.96 is specific for the 95% confidence interval, there exist other multipliers for other probabilities. Some of them are presented in the table below:

Percentage	Multiplier
50	0.67
75	1.15
80	1.28
90	1.64
95	1.96
99	2.58

One step prediction intervals

- In order to compute the prediction interval, we need the standard deviation of the forecast distribution.
- When we forecast only one step ahead (i.e., $h = 1$), this can be estimated by the standard deviation of the residuals:

$$\hat{\sigma} = \sqrt{\frac{1}{T - K - M} * \sum_{t=1}^{T} e_t^2}$$

- where K is the number of parameters estimated in the forecasting method and M is the number of missing values in the residuals (observations for which we do not have a forecast).

- Let's see this in practice for the naive model on the Google 2015 stock price data used previously.
  - The last observation (day 252) had a value of 759, so according to the naive model, this is going to be the 1 step forecast value.
  - In order to get the prediction interval, we need the standard deviation (the formula on the previous slide), which is 11.19.
  - The 95% prediction interval is then $758.88 \pm 1.96 * 11.19$ which is $[736.95, 780.81]$
  - The 80% prediction interval is then $758.88 \pm 1.28 * 11.19$ which is $[744.78, 772.98]$

Multi-step prediction intervals

- In general, if the prediction horizon increases, so does the length of the prediction interval $\rightarrow$ the further away we forecast, the more uncertainty we have.
- For computing the prediction interval for an h-step forecast, we need the value of σ_h. We have seen how to compute σ_1, for larger values the computations are more complicated.

- Assuming that the residuals are not correlated, the value of σ_h was mathematically determined for the four benchmark methods:

Method	h-step forecast standard deviation
Mean	$\hat{\sigma}_h = \hat{\sigma}_1 * \sqrt{1 + 1/T}$
Naive	$\hat{\sigma}_h = \hat{\sigma}_1 * \sqrt{h}$
Seasonal naive	$\hat{\sigma}_h = \hat{\sigma}_1 * \sqrt{k + 1}$
Drift	$\hat{\sigma}_h = \hat{\sigma}_1 * \sqrt{h * (1 + h/(T - 1))}$

- where m is the seasonal period and k is the integer part of $\frac{h-1}{m}$ (i.e. the number of complete years in the forecast period, prior to the time $T + h$).

- We can get the prediction intervals in R using the *hilo* function which, by default displays the 80% and 95% prediction interval.
- If you want to get prediction interval for another percentage, it can be set with the *level* argument.
- The *level* argument can be used in the *autoplot* function as well, to specify what prediction interval to display (or, by setting it to *NULL*, we can specify that we do not want the prediction interval displayed).

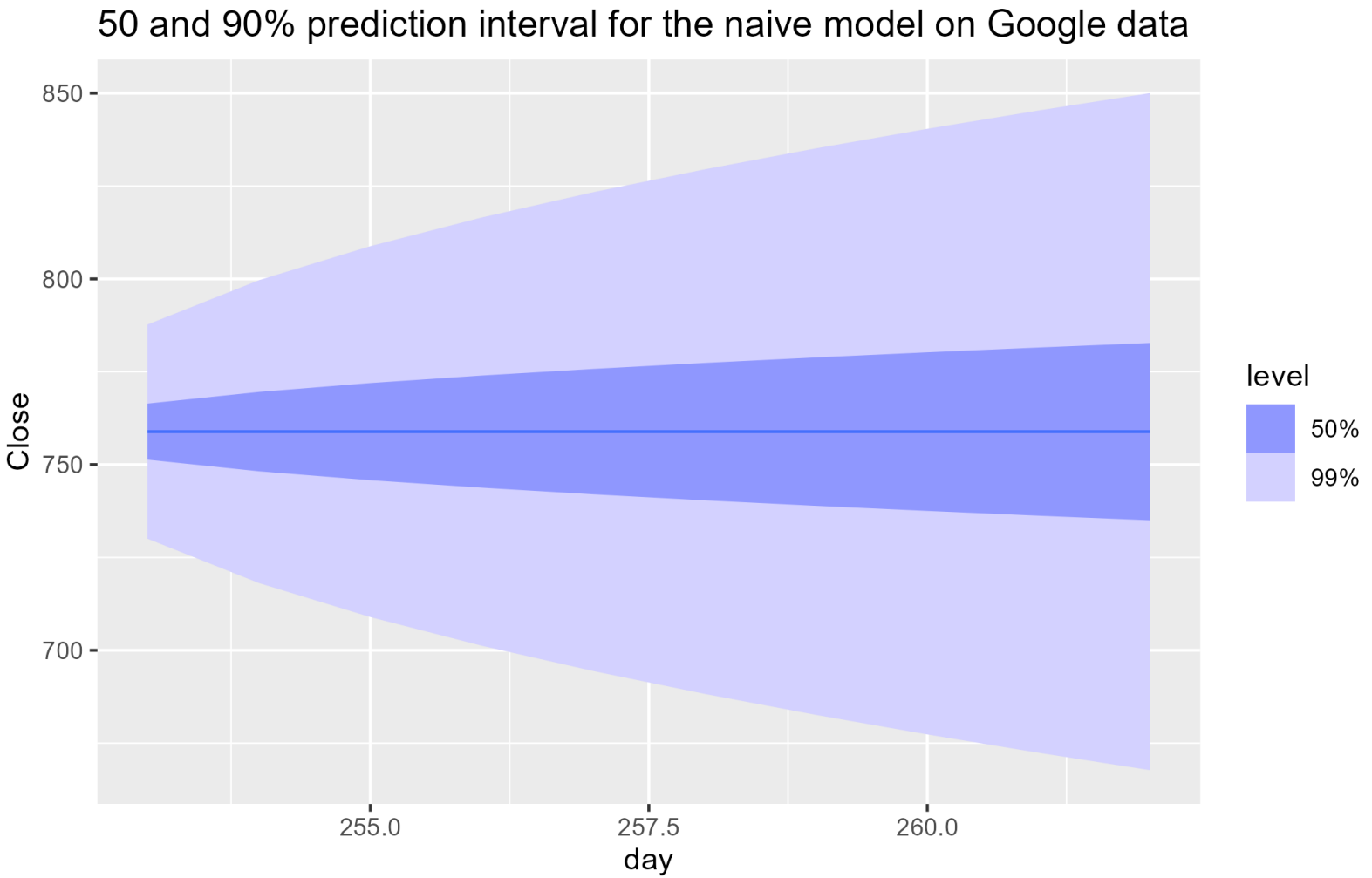
```
google2015Data |>
    model(NAIVE(Close)) |>
    forecast(h = 10) |>
    hilo()
```

```
.model          day        Close .mean                 `80%`                  `95%`
  <chr>         <dbl>       <dist> <dbl>                <hilo>                 <hilo>
1 NAIVE(Close)   253  N(759, 125)  759. [744.5400, 773.2200]80 [736.9488, 780.8112]95
2 NAIVE(Close)   254  N(759, 250)  759. [738.6001, 779.1599]80 [727.8646, 789.8954]95
3 NAIVE(Close)   255  N(759, 376)  759. [734.0423, 783.7177]80 [720.8941, 796.8659]95
4 NAIVE(Close)   256  N(759, 501)  759. [730.1999, 787.5601]80 [715.0176, 802.7424]95
5 NAIVE(Close)   257  N(759, 626)  759. [726.8147, 790.9453]80 [709.8404, 807.9196]95
6 NAIVE(Close)   258  N(759, 751)  759. [723.7543, 794.0058]80 [705.1598, 812.6002]95
7 NAIVE(Close)   259  N(759, 876)  759. [720.9399, 796.8202]80 [700.8556, 816.9045]95
8 NAIVE(Close)   260 N(759, 1002)  759. [718.3203, 799.4397]80 [696.8493, 820.9108]95
9 NAIVE(Close)   261 N(759, 1127)  759. [715.8599, 801.9001]80 [693.0865, 824.6735]95
10 NAIVE(Close)   262 N(759, 1252)  759. [713.5329, 804.2272]80 [689.5275, 828.2325]95
```

```
google2015Data |>
    model(NAIVE(Close)) |>
    forecast(h = 10) |>
    hilo(level = 99)
```

```
 .model          day        Close .mean                  `99%`
   <chr>         <dbl>       <dist> <dbl>                 <hilo>
 1 NAIVE(Close)   253  N(759, 125)  759. [730.0575, 787.7025]99
 2 NAIVE(Close)   254  N(759, 250)  759. [718.1189, 799.6411]99
 3 NAIVE(Close)   255  N(759, 376)  759. [708.9580, 808.8020]99
 4 NAIVE(Close)   256  N(759, 501)  759. [701.2351, 816.5249]99
 5 NAIVE(Close)   257  N(759, 626)  759. [694.4310, 823.3290]99
 6 NAIVE(Close)   258  N(759, 751)  759. [688.2797, 829.4803]99
 7 NAIVE(Close)   259  N(759, 876)  759. [682.6230, 835.1371]99
 8 NAIVE(Close)   260 N(759, 1002)  759. [677.3578, 840.4022]99
 9 NAIVE(Close)   261 N(759, 1127)  759. [672.4126, 845.3474]99
10 NAIVE(Close)   262 N(759, 1252)  759. [667.7354, 850.0246]99
```

```
google2015Data |>
    model(NAIVE(Close)) |>
    forecast(h = 10) |>
    autoplot(level = c(50, 99)) +
    labs(title = "50 and 90% prediction interval for the Naive model on Google
     data")
```


50 and 90% prediction interval for the naive model on Google data
850
800
750
700
Close
255.0
257.5
260.0
day
level
50%
99%

Forecasting using transformations

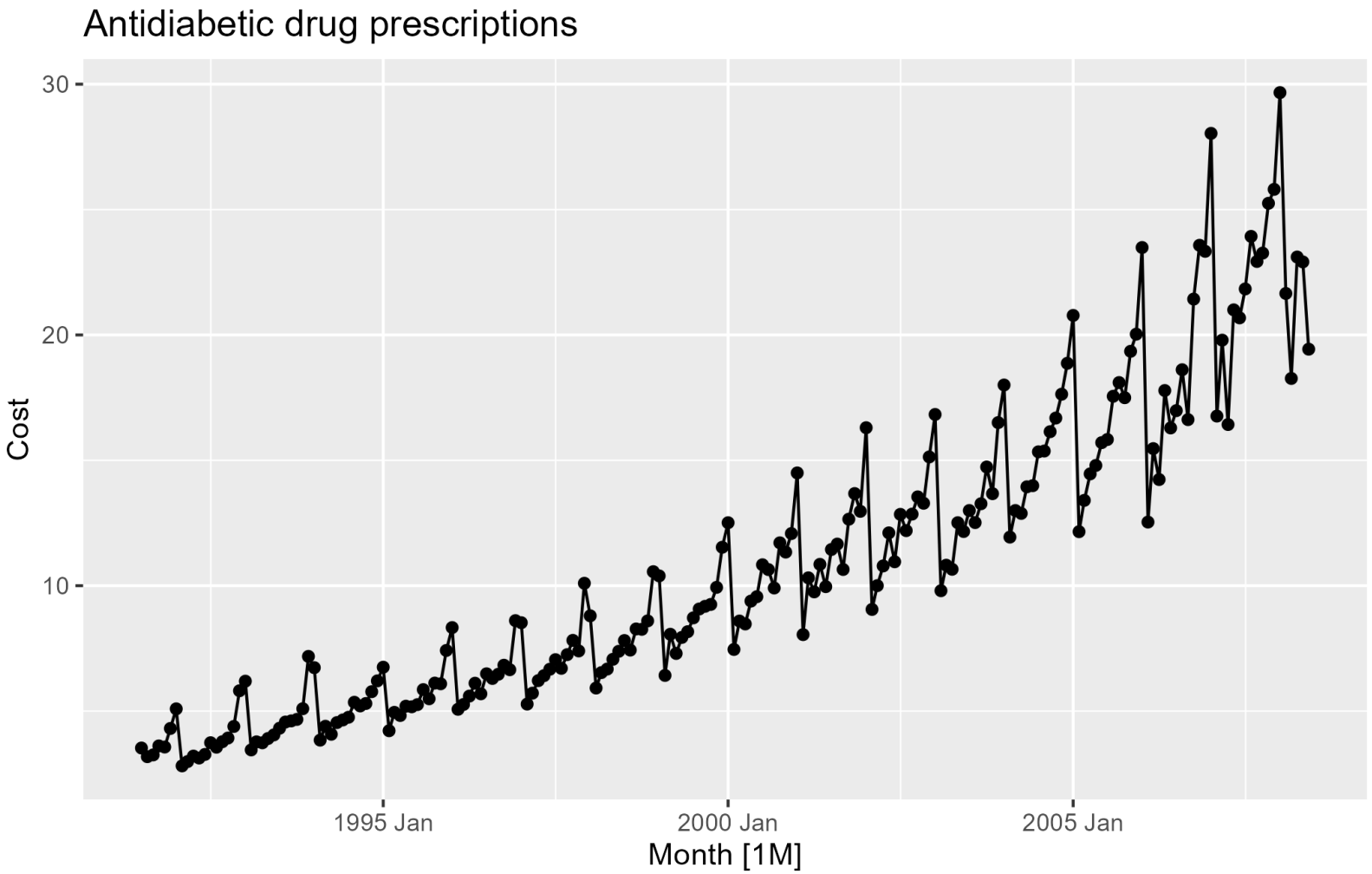
- We have talked about performing transformations on the data before forecasting (for example the log transformation, or Box-Cox to stabilize variance in the data).
- When working with transformed data, the forecasts and the prediction intervals will be on the transformed scale, so we need to *back-transform* them to obtain values on the original scale.
- Transformations will not influence the point forecast that much, but they have a great influence on the prediction interval: after the back-transformation the prediction interval will no longer be symmetric around the point.
- In R if we specify the transformation when specifying the model, it will take care automatically of the transformation and back-transformation.
- Using the *augment* function we can check the residuals and the innovative residuals.

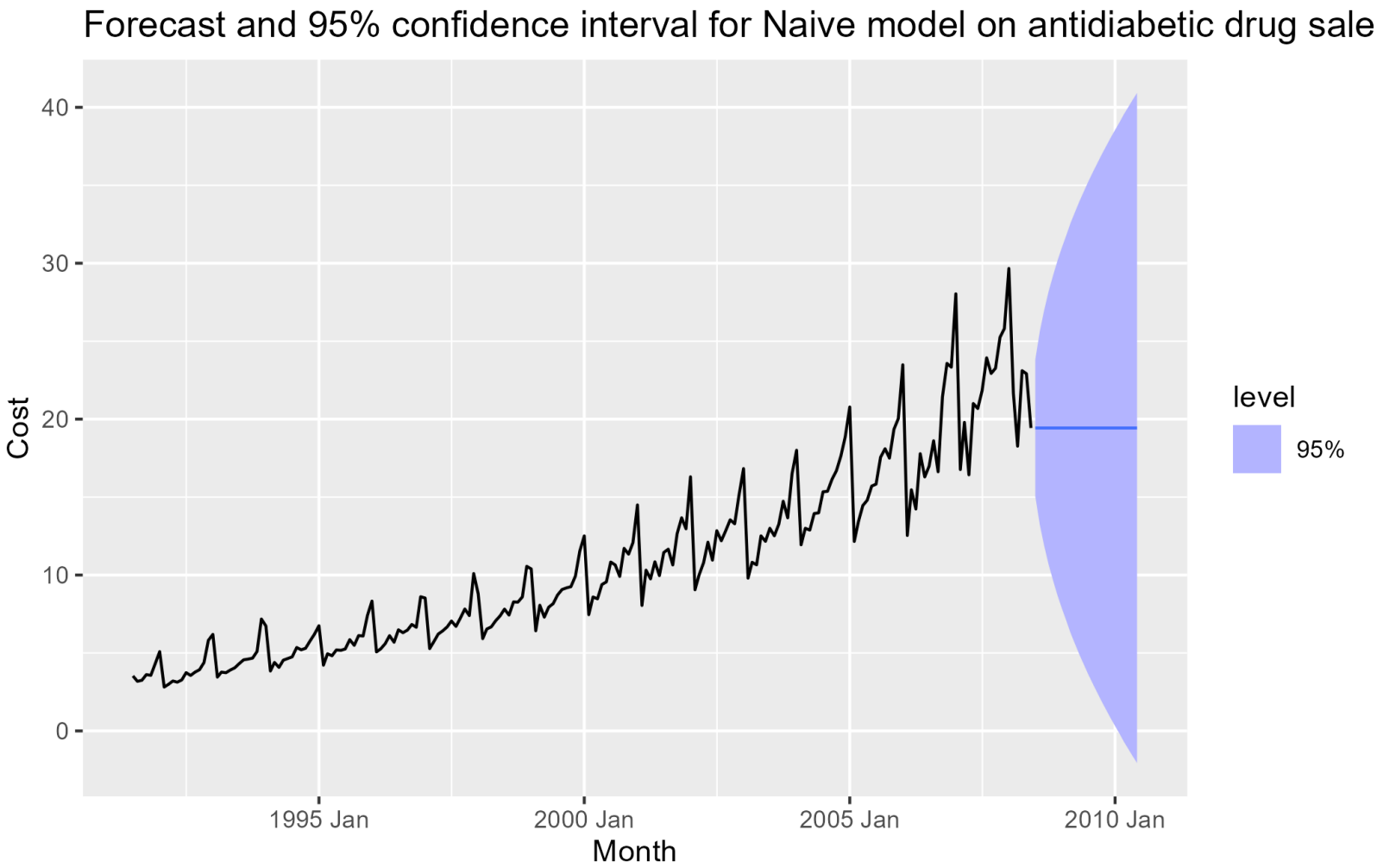
```
antidiabetic <- PBS |>
  filter(ATC2 == "A10") |>
  select(Month, Concession, Type, Cost) |>
  summarize(TotalCost = sum(Cost)) |>
  mutate(Cost = TotalCost / 1000000) #just to have smaller values
antidiabetic |> autoplot(Cost) +
  labs(title = "Antidiabetic drug prescriptions")

antidiabetic |>
  model(naive = NAIVE(Cost)) |>
  forecast(h = 24) -> antiDForecast

antiDForecast |>
  autoplot(level = 95) +
  autolayer(antidiabetic, Cost) +
  labs(title = "Forecast and 95% confidence interval for Naive model on
     antidiabetic drug sales data")

antiDForecast |> hilo()
```


Antidiabetic drug prescriptions
30
20
10
Cost
1995 Jan
2000 Jan
2005 Jan
Month [1M]


Forecast and 95% confidence interval for Naive model on antidiabetic drug sale
Cost
40
30
20
10
0
1995 Jan
2000 Jan
2005 Jan
2010 Jan
Month
level
95%

	.model	Month	Cost	.mean	‘80%‘	‘95%‘
	<chr>	<mth>	<dist>	<dbl>	<hilo>	<hilo>
1	naive	2008 Jul	N(19, 5)	19.4	[16.56374, 22.29974]80	[15.045511, 23.81797]95
2	naive	2008 Aug	N(19, 10)	19.4	[15.37577, 23.48771]80	[13.228676, 25.63480]95
3	naive	2008 Sep	N(19, 15)	19.4	[14.46422, 24.39926]80	[11.834569, 27.02891]95
4	naive	2008 Oct	N(19, 20)	19.4	[13.69574, 25.16774]80	[10.659283, 28.20420]95
5	naive	2008 Nov	N(19, 25)	19.4	[13.01870, 25.84478]80	[9.623835, 29.23965]95
6	naive	2008 Dec	N(19, 30)	19.4	[12.40660, 26.45688]80	[8.687718, 30.17576]95
7	naive	2009 Jan	N(19, 35)	19.4	[11.84372, 27.01976]80	[7.826870, 31.03661]95
8	naive	2009 Feb	N(19, 40)	19.4	[11.31981, 27.54367]80	[7.025612, 31.83787]95
9	naive	2009 Mar	N(19, 45)	19.4	[10.82774, 28.03574]80	[6.273054, 32.59043]95
10	naive	2009 Apr	N(19, 50)	19.4	[10.36233, 28.50115]80	[5.561267, 33.30221]95

- Let's try to build a naive model on the log-transformed observation values

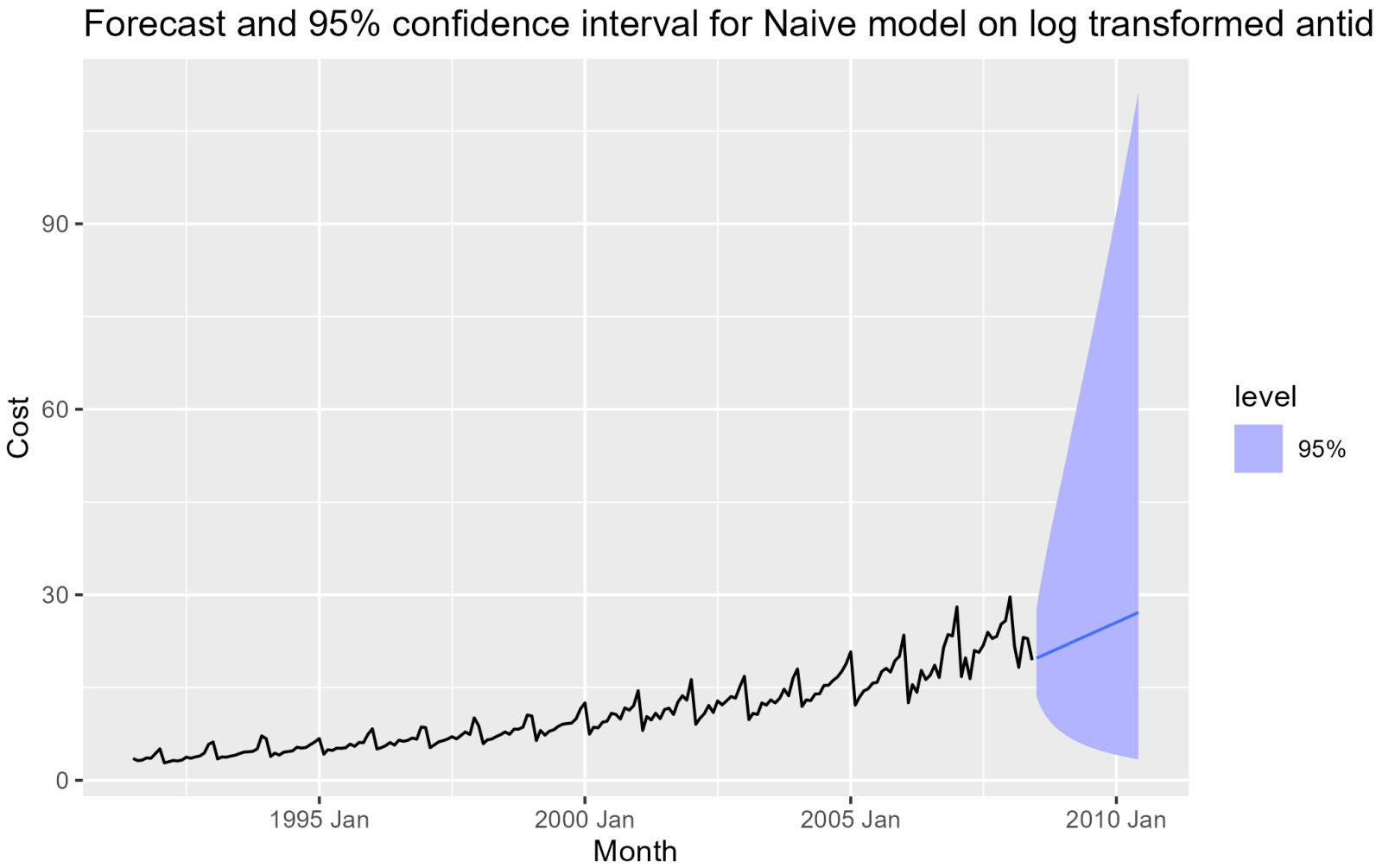
```
antidiabetic |>
  model(naive = NAIVE(log(Cost))) |>
  forecast(h= 24) -> antiDLogForecast
antidiabetic |>
  model(naive = NAIVE(log(Cost))) |>
  augment()
```

	.model	Month	Cost	.fitted	.resid	.innov
	<chr>	<mth>	<dbl>	<dbl>	<dbl>	<dbl>
1	naive	1991 Jul	3.53	NA	NA	NA
2	naive	1991 Aug	3.18	3.53	-0.346	-0.103
3	naive	1991 Sep	3.25	3.18	0.0713	0.0222
4	naive	1991 Oct	3.61	3.25	0.359	0.105
5	naive	1991 Nov	3.57	3.61	-0.0451	-0.0126
6	naive	1991 Dec	4.31	3.57	0.741	0.189
7	naive	1992 Jan	5.09	4.31	0.782	0.167
8	naive	1992 Feb	2.81	5.09	-2.27	-0.592
9	naive	1992 Mar	2.99	2.81	0.171	0.0591
10	naive	1992 Apr	3.20	2.99	0.219	0.0708

```
antiDLogForecast |> hilo()
```

```
   .model    Month           Cost .mean                 ‘80%‘                  ‘95%‘
   <chr>     <mth>          <dist> <dbl>               <hilo>                 <hilo>
 1 naive  2008 Jul t(N(3, 0.033))  19.8 [15.394061, 24.52845]80 [13.608277, 27.74727]95
 2 naive  2008 Aug t(N(3, 0.066))  20.1 [13.978224, 27.01291]80 [11.741424, 32.15901]95
 3 naive  2008 Sep t(N(3, 0.099))  20.4 [12.980758, 29.08863]80 [10.484521, 36.01428]95
 4 naive  2008 Oct  t(N(3, 0.13))  20.7 [12.195363, 30.96198]80 [ 9.530037, 39.62131]95
 5 naive  2008 Nov  t(N(3, 0.17))  21.0 [11.542887, 32.71214]80 [ 8.761387, 43.09735]95
 6 naive  2008 Dec   t(N(3, 0.2))  21.4 [10.983102, 34.37941]80 [ 8.119975, 46.50169]95
 7 naive  2009 Jan  t(N(3, 0.23))  21.7 [10.492319, 35.98752]80 [ 7.571664, 49.86916]95
 8 naive  2009 Feb  t(N(3, 0.26))  22.0 [10.055236, 37.55183]80 [ 7.094631, 53.22229]95
 9 naive  2009 Mar   t(N(3, 0.3))  22.3 [ 9.661315, 39.08293]80 [ 6.673997, 56.57667]95
10 naive  2009 Apr  t(N(3, 0.33))  22.6 [ 9.302945, 40.58849]80 [ 6.299126, 59.94364]95
```

```
antiDLogForecast |>
  autoplot(level = 95) +
  autolayer(antidiabetic, Cost) +
  labs(title = "Forecast and 95% confidence interval for Naive model on log
     transformed antidiabetic drug sales data")
```

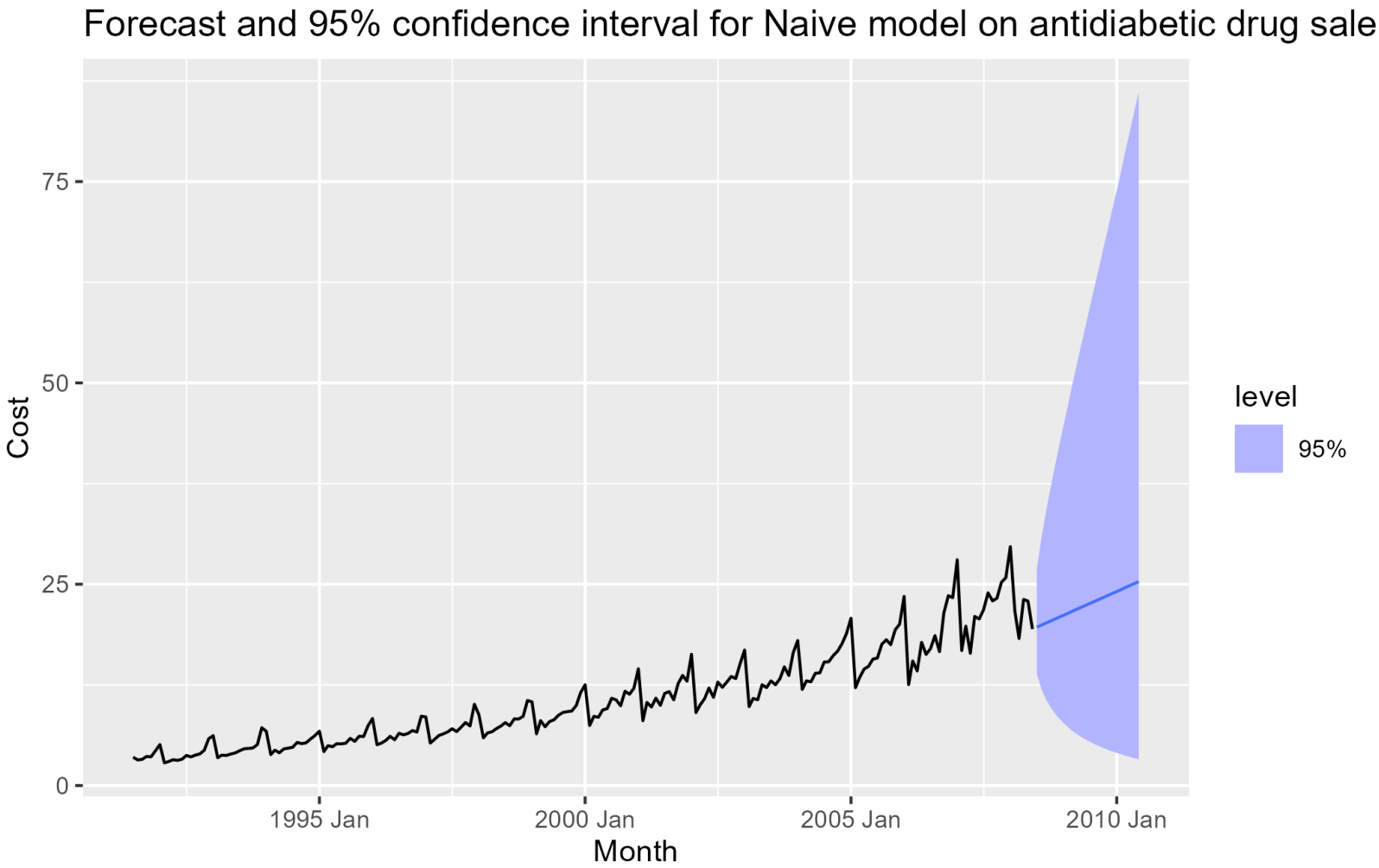

Forecast and 95% confidence interval for Naive model on log transformed antid
Cost
90
60
30
0
1995 Jan
2000 Jan
2005 Jan
2010 Jan
Month
level
95%

- And build a naive model on the data after a Box-Cox transformation

```
antidiabetic |>
  model(naive = NAIVE(box_cox(Cost, lambda = 0.11))) |>
  forecast(h= 24) -> antiDBCForecast
antiDBCForecast |> hilo()
```

	.model	Month	Cost	.mean	‘80%‘	‘95%‘
	<chr>	<mth>	<dist>	<dbl>	<hilo>	<hilo>
1	naive	2008 Jul	t(N(3.5, 0.054))	19.7	[15.618190, 24.05282]80	[13.882422, 26.87496]95
2	naive	2008 Aug	t(N(3.5, 0.11))	19.9	[14.244655, 26.23715]80	[12.032392, 30.63684]95
3	naive	2008 Sep	t(N(3.5, 0.16))	20.2	[13.264726, 28.03116]80	[10.765130, 33.83394]95
4	naive	2008 Oct	t(N(3.5, 0.22))	20.4	[12.485711, 29.62753]80	[9.790574, 36.75638]95
5	naive	2008 Nov	t(N(3.5, 0.27))	20.7	[11.833408, 31.10049]80	[8.997887, 39.51504]95
6	naive	2008 Dec	t(N(3.5, 0.33))	20.9	[11.269973, 32.48786]80	[8.330955, 42.16608]95
7	naive	2009 Jan	t(N(3.5, 0.38))	21.1	[10.773048, 33.81201]80	[7.756863, 44.74248]95
8	naive	2009 Feb	t(N(3.5, 0.44))	21.4	[10.328143, 35.08745]80	[7.254434, 47.26545]95
9	naive	2009 Mar	t(N(3.5, 0.49))	21.6	[9.925250, 36.32417]80	[6.809136, 49.74958]95
10	naive	2009 Apr	t(N(3.5, 0.54))	21.9	[9.557119, 37.52944]80	[6.410522, 52.20536]95

```
antiDBCForecast |>
  autoplot(level = 95) +
  autolayer(antidiabetic, Cost) +
  labs(title = "Forecast and 95% confidence interval for Naive model on
      antidiabetic drug sales data with a Box–Cox transformation")
```


Forecast and 95% confidence interval for Naive model on antidiabetic drug sale
Cost
75
50
25
0
1995 Jan
2000 Jan
2005 Jan
2010 Jan
Month
level
95%

Forecasting with decomposition

- We have talked about ways to decompose a time series into three components: trend, seasonal and remainder. We can use decomposition before forecasting: decompose the data, forecast the components separately, then combine the results.
- In order to do this, we will consider that the time series is made of two components only: the seasonal component ($\hat{S}_t$) and the seasonally adjusted components ($\hat{A}_t$), which contains the trend and the remainder.
- So we have:

$$y_t = \hat{S}_t + \hat{A}_t \text{ where } \hat{A}_t = \hat{T}_t + \hat{R}_t$$

- or

$$y_t = \hat{S}_t * \hat{A}_t \text{ where } \hat{A}_t = \hat{T}_t * \hat{R}_t$$

- Since the seasonal component in general changes slowly in time, it can be forecast with a seasonal naive model.
- The seasonally adjusted component can be forecast with any non-seasonal forecasting method, for example the drift method (or more complex methods that will be discussed later).
- Once you have the forecasts for the components, the prediction interval can be computed by adding the prediction intervals of the forecast components.
- Let's take an example, where we use as data the *us_employment* information from 1990 in the retail trade.

```r
us_retail_employment <- us_employment |>
  filter(year(Month)>= 1990, Title == "Retail Trade") |>
  select(Month, Employed)

dcmp <- us_retail_employment |>
  model(STL(Employed ~ trend(window = 7), robust = TRUE)) |>
  components()
dcmp

dcmp |> autoplot() +
  labs(title = "STL decomposition of the retail trade employment data")

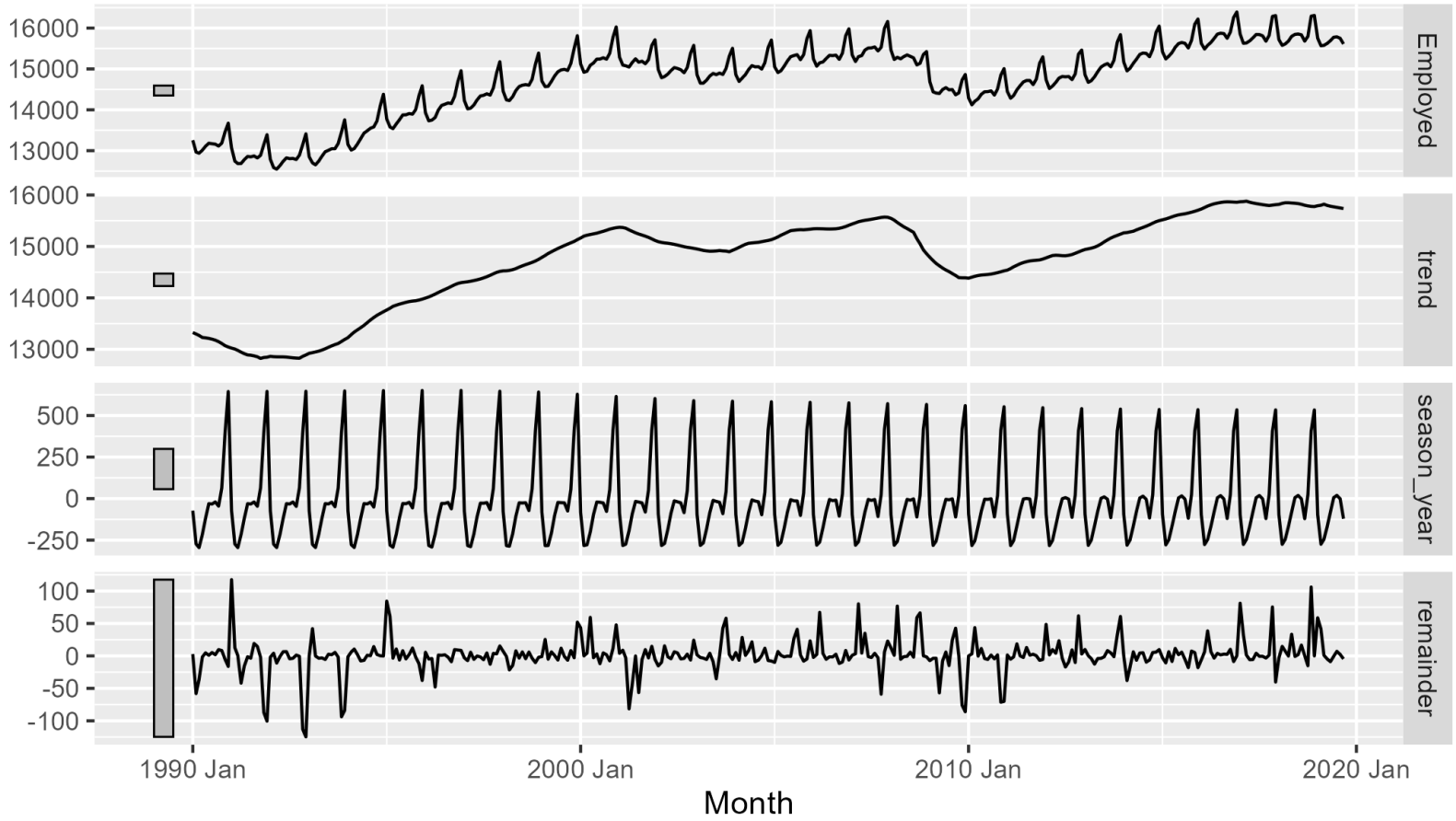
dcmp |> as_tibble() |>
  ggplot() +
  geom_line(mapping = aes(x = Month, y = season_adjust)) +
  labs(title= "Seasonally adjusted component for US retail employment data")

dcmp |>
  select(-.model) |> #should not have attribute called .model
  model(NAIVE(season_adjust)) |>
  forecast(h = 24) -> dcmpForecast

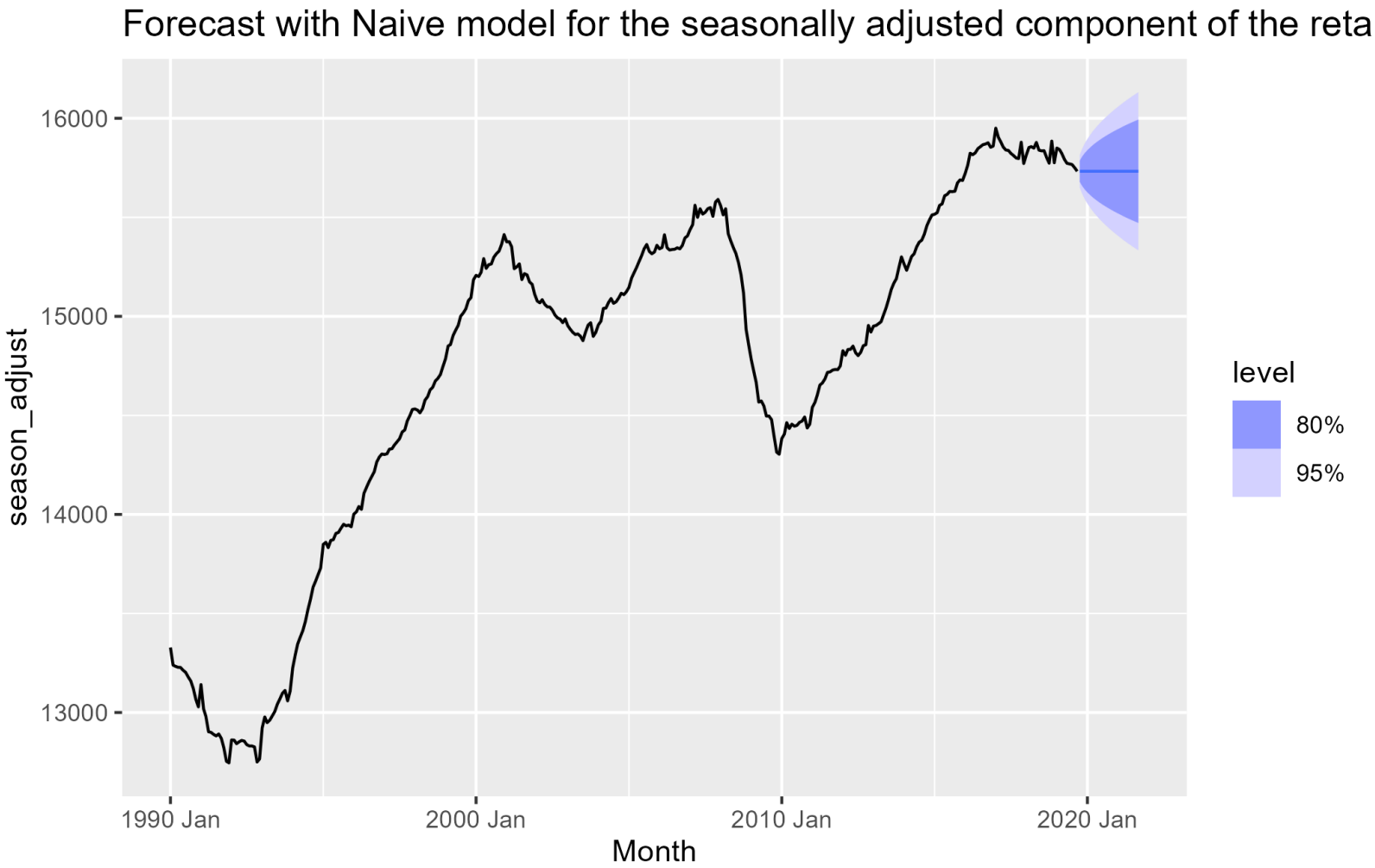
dcmpForecast |>
  autoplot() +
  autolayer(dcmp, season_adjust) +
  labs(title = "Forecast with Naive model for the seasonally adjusted component
     of the retail trade employment data")
```

STL decomposition of the retail trade employment data

Employed = trend + season_year + remainder

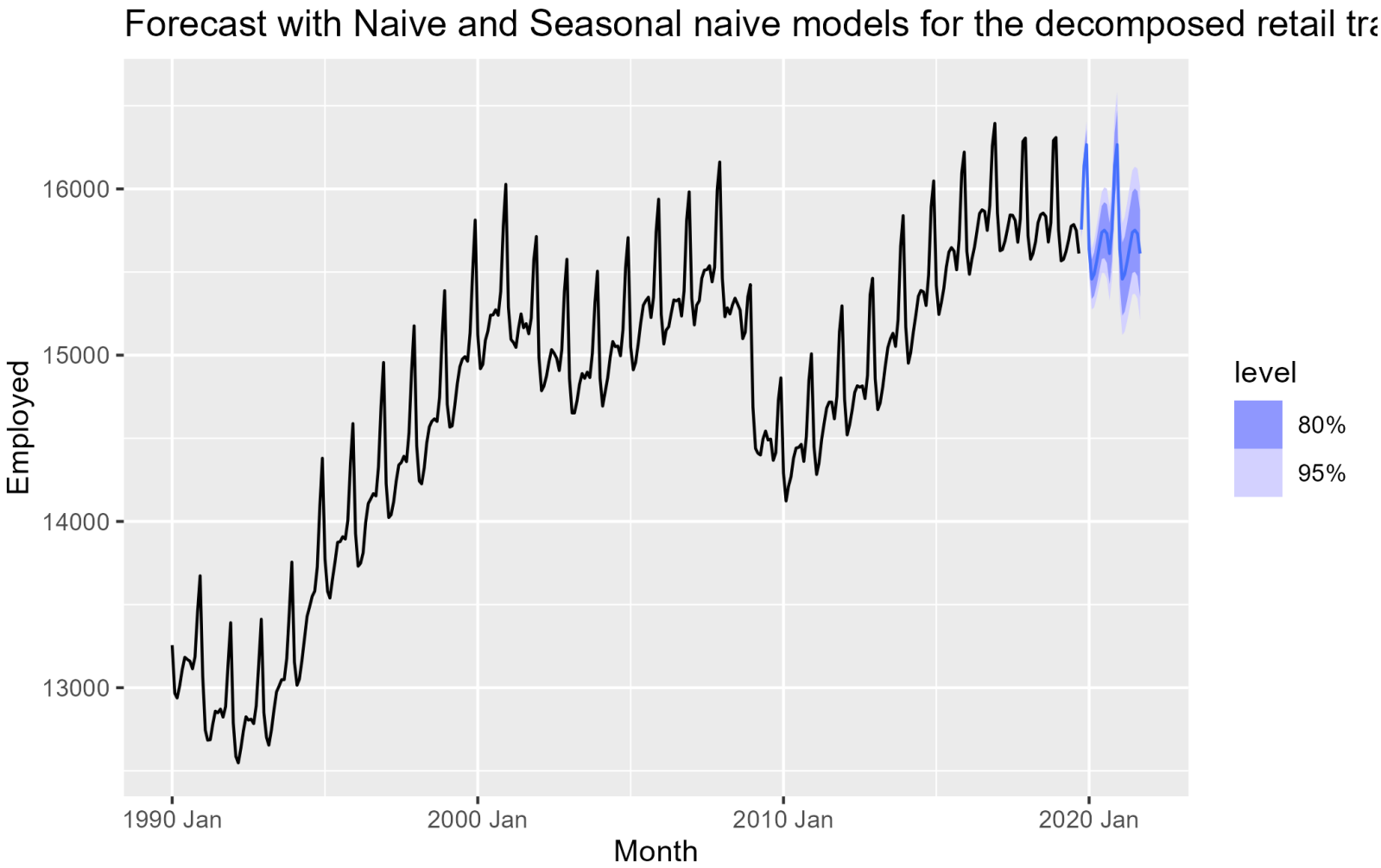



Seasonally adjusted component for US retail employment data
16000
15000
14000
13000
season_adjust
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month


Forecast with Naive model for the seasonally adjusted component of the reta
16000
15000
14000
13000
season_adjust
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month
level
80%
95%

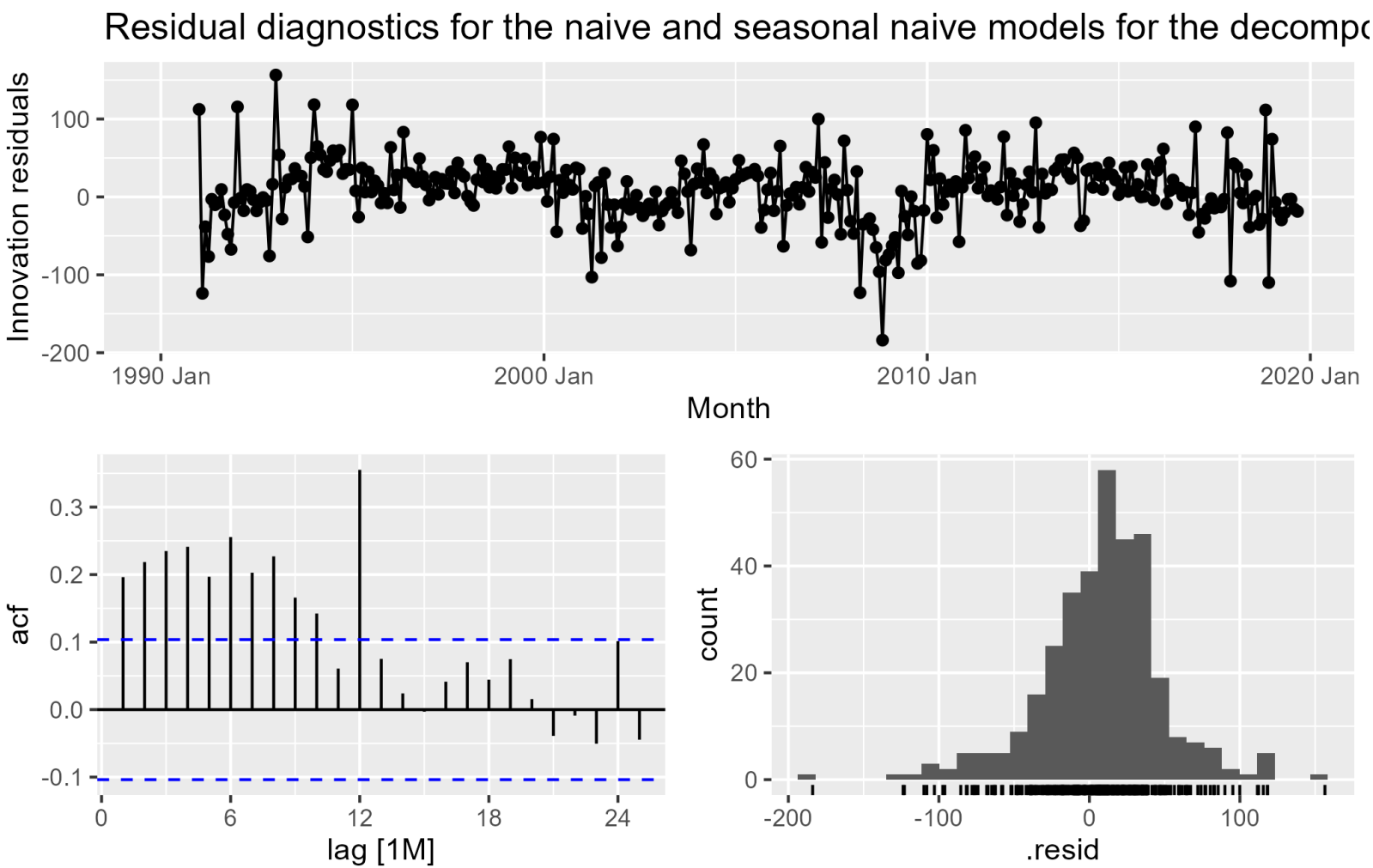
- In the above code we have decomposed the time series and created a naive model for the seasonally adjusted component. But we actually want to combine two models (one for the seasonal data and one for the seasonally adjusted data). This can be done easily with the *decomposition_model* function, which allows you to compute the forecasts using any additive decomposition, using other models of functions to forecast the components. The function will also combine the forecasted values of the components.

```
forecast_dcmp <- us_retail_employment |>
  model(stlf = decomposition_model(STL(Employed ~trend(window = 7), robust =
      TRUE),
         NAIVE(season_adjust),
         SNAIVE(season_year ~ lag("year"))))
forecast_dcmp
forecast_dcmp |>
  forecast(h = 24) -> dcmp2Forecast
dcmp2Forecast |>
  autoplot() +
  autolayer(us_retail_employment, Employed) +
  labs(title = "Forecast with Naive and Seasonal naive models for the decomposed
       retail trade employment data")
```


Forecast with Naive and Seasonal naive models for the decomposed retail tra
Employed
16000
15000
14000
13000
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month
level
80%
95%

- The seasonal naive model might be omitted, if no other model is specified the *decomposition_model* will automatically use SNAIVE to forecast the seasonal component.
- Let's look at the residuals of the model, to see if this is a good model

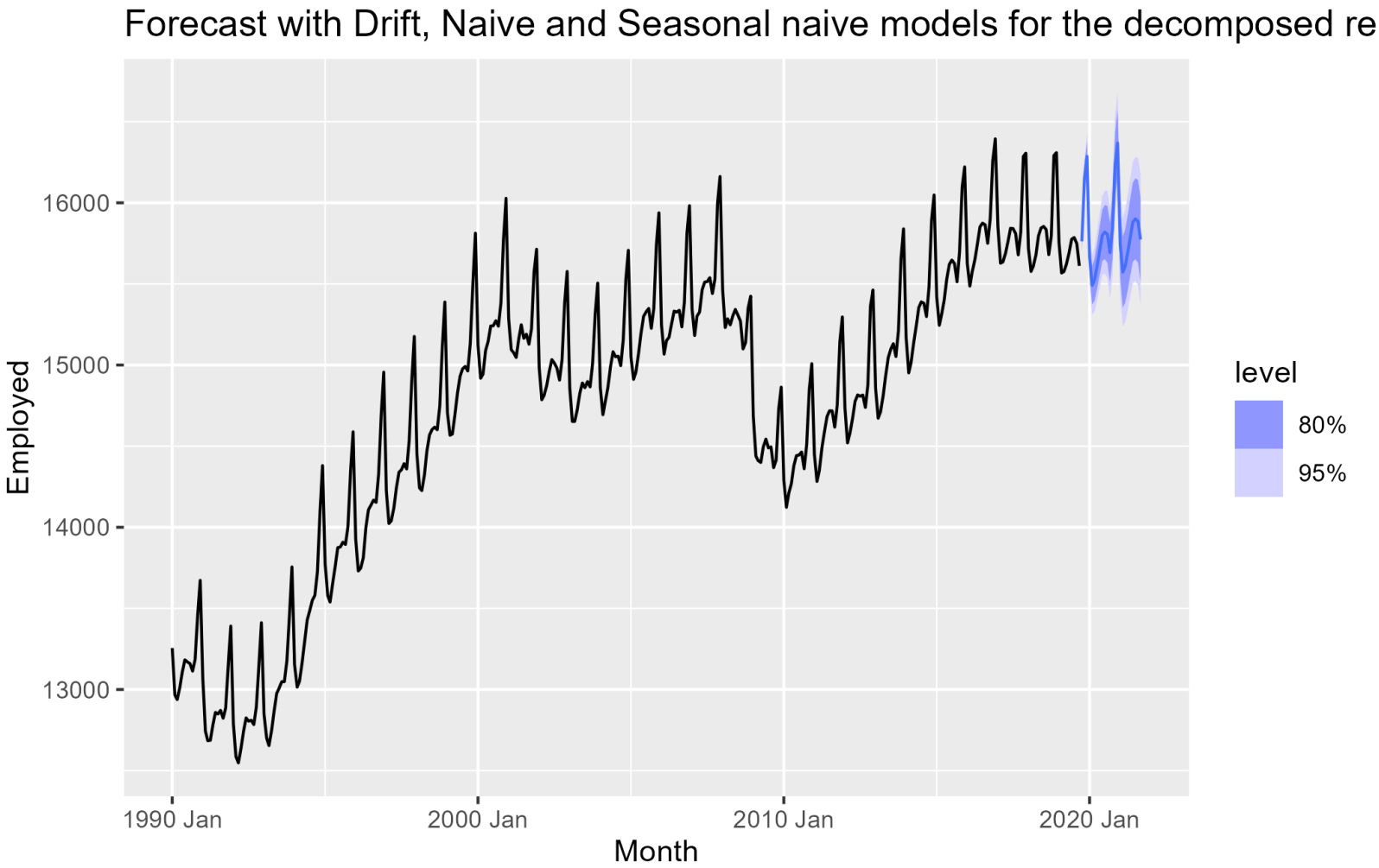
```
forecast_dcmp |> gg_tsresiduals() +
  labs(title = "Residual diagnostics for the naive and seasonal naive models for
      the decomposed retail trade employment data")
```

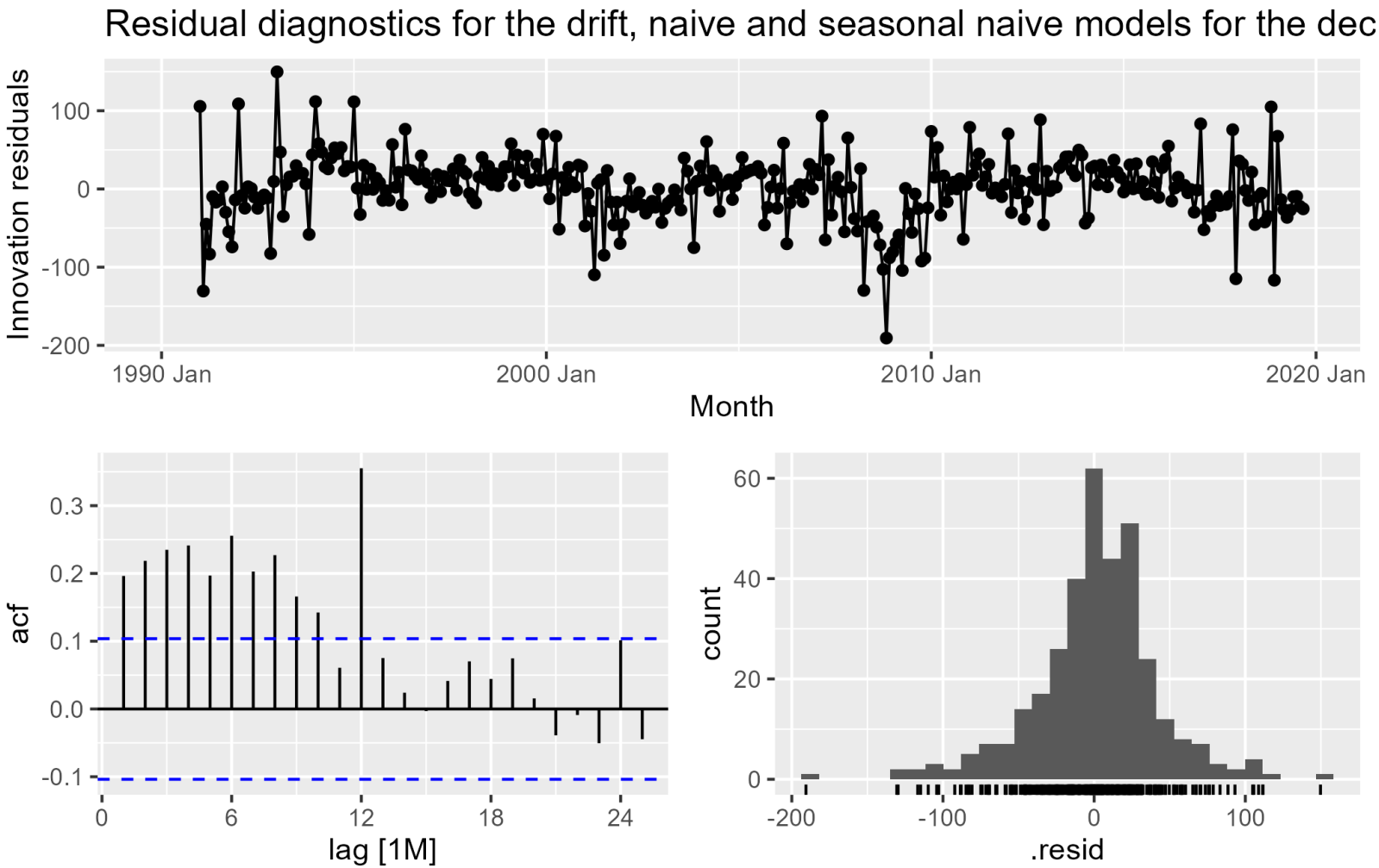

Residual diagnostics for the naive and seasonal naive models for the decompo
Innovation residuals
100
0
-100
-200
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month
acf
0.3
0.2
0.1
0.0
-0.1
0
6
12
18
24
lag [1M]
count
60
40
20
0
-200
-100
0
100
.resid

- That is a lot of autocorrelation, suggesting that this is not a good model.
- Let's try to forecast separately all three components of the time series: the seasonal component with SNAIVE, the trend with the drift and the remainder with the NAIVE component.

```
forecast_dcmp2 <- us_retail_employment |>
  model(stlf = decomposition_model(STL(Employed ~trend(window = 7), robust =
      TRUE),
         RW(trend ~ drift()),
         NAIVE(remainder),
         SNAIVE(season_year ~ lag("year"))))
forecast_dcmp2 |>
  forecast(h = 24) -> dcmp2Forecast2
dcmp2Forecast2 |>
  autoplot() +
  autolayer(us_retail_employment, Employed) +
  labs(title = "Forecast with Drift, Naive and Seasonal naive models for the
      decomposed retail trade employment data")

forecast_dcmp2 |> gg_tsresiduals() +
  labs(title = "Residual diagnostics for the drift, naive and seasonal naive
      models for the decomposed retail trade employment data")
```


Forecast with Drift, Naive and Seasonal naive models for the decomposed re
Employed
16000
15000
14000
13000
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month
level
80%
95%


Residual diagnostics for the drift, naive and seasonal naive models for the dec
Innovation residuals
100
0
-100
-200
1990 Jan
2000 Jan
2010 Jan
2020 Jan
Month
acf
0.3
0.2
0.1
0.0
-0.1
0
6
12
18
24
lag [1M]
count
60
40
20
0
-200
-100
0
100
.resid

Evaluating the accuracy of the forecast

- Residuals are computed for the data that is used to build the model. Their value is not a good indicator of true performance. For evaluation we need data that is not used for building the model $\rightarrow$ we split our data in *training* (used to estimate parameters and build the model) and *testing* (used for evaluating the model).
- The *test data*:
  - Should be at least as large as the maximum forecast horizon required.
  - In general is about 20% of the data (but this depends on how much data you have).
- **Obs:** Since we are talking about time series, the order of observations is important, we simply *cut* the time series at the required point to split in training and testing.

- Similar to other domains where models are trained and predictions are made (ex. Machine Learning), in case of forecasting we have the following observations as well:
  - A model which performs well on training data will not necessarily have good performance on unseen test data.
  - You can always achieve good performance on your training data, if you use a model which is complex enough (overfitting)
  - Overfitting the training data is just as bad as not identifying real patterns in it.

- Functions that can be used to split data:
  - *filter* - you can specify conditions on the time index (ex. only data until year 2000; only data starting from 2000, etc.)
  - *slice* - allows the usage of indexes.
  - *head* and *tail*
- Consider the *ausbeer* data set containing information about quarterly beer production in Australia from 1956 to 2010 (only 2 quarters from 2010).

```
aus_beer <- aus_production |> select(Beer)
aus_beer |>
  filter(year(Quarter) < 1995)
aus_beer |>
  filter(year(Quarter) >= 1995)

aus_beer |> slice(0:100)
aus_beer |> slice(-1)
aus_beer |> slice(n())
aus_beer |> slice(5: n())
aus_beer |> slice(-200:0)
aus_beer |> slice(n()-19: 0)

trainSize <- dim(aus_beer)[1] * 0.8
testSize <- dim(aus_beer)[1] * 0.2
trainData <- aus_beer |> head(trainSize)
testData <- aus_beer |> tail(testSize)
trainData |> tail()
testData
```

Forecast error

- The *forecast error* is the difference between the predicted and the actual value. Considering that the training data is $\{y_1, \ldots, y_T\}$ and the testing data is $\{y_{T+1}, y_{T+2} \ldots\}$ the error is $e_{T+h} = y_{T+h} - \hat{y}_{T+h}$
- The error of the prediction model can be computed by aggregating the forecast errors in different ways.
- **Obs:** Forecast error is not the same as the residual, for two reasons:
  - The residual is something you compute on the training data, while the error is computed on the test data
  - The residual always refers to one-step prediction, while in case of the error we can have multiple steps.

Scale dependent measures

- Two popular measures of forecast accuracy are: *Mean absolut error (MAE)* and *Root mean squared error (RMSE)* computed in the following way:

$$MAE = mean(|e_t|)$$

$$RMSE = \sqrt{mean(e_t^2)}$$

- Both MAE and RMSE are measures that should be minimised. A method that minimizes MAE will forecast the median, while a method minimizing the RMSE will forecast the mean.
- **Obs:** e_t, MAE and RMSE use the same units of measurement (scale) as the input data, so they are NOT suitable for comparing results on different data sets. They can be used to compare the performance of different models on the same data set or performance on different data sets using the same units of measurement.

Percentage errors

- The percentage error can be computed by $p_t = 100 \times \frac{e_t}{y_t}$.
- One measure built based on the percentage error is the *Mean absolute percentage error (MAPE)* which is computed in the following way:

$$MAPE = mean(|p_t|)$$

- The advantage of MAPE is that it can be used to compare results across different data sets.
- However, it has a few disadvantages:
  - You have a problem when y_t is 0 (error is either infinite or undefined).
  - You might also have a problem when y_t is close to zero (p_t being extremely small or large).
  - It assumes that the unit of measurement has a meaningful zero value.

Symmetric MAPE

- Another disadvantage of MAPE is that it has no upper bounds for over - predictions. Consider the following situation:
  - $y_t = 10$ and $\hat{y}_t = 0$ (under-forecast). In this case MAPE = 100%. Assuming data with positive values, $\hat{y}_t$ cannot be lower than that. However, in case of an over-forecast, there is no limit how big the error can get: if $\hat{y}_t = 100$, MAPE = 900%.
  - This property is also the reason why blindly optimizing MAPE of a model, might lead to a model which always under-forecasts (which might not be desirable in certain domains, for ex. sales).
- This is why *sMAPE* (symmetric MAPE) was proposed:

$$sMAPE = mean(200 * \frac{|y_t - \hat{y}_t|}{y_t + \hat{y}_t})$$

- sMAPE still suffers from the division by a number close to 0. And it can be negative.

Scaled errors

- An alternative for the percentage error is based on scaling the error considering the *training MAE* from a simple forecast method.
- In case of non-seasonal data, we can use the naive method as a simple forecast method and define the scaled error q_j in the following way:

$$q_j = \frac{e_j}{\frac{1}{T-1}\sum_{t=2}^{T}|y_t - y_{t-1}|}$$

- q_j is less than one if it comes from a better forecast than the average one-step naive method on the training data.
- In case of seasonal data, the seasonal naive method can be used as a simple forecast method for defining q_j:

$$q_j = \frac{e_j}{\frac{1}{T-m}\sum_{t=m+1}^{T}|y_t - y_{t-m}|}$$

- Then, the *mean absolute scaled error (MASE)* can be computed as:

$$MASE = mean(|q_j|)$$

- MASE is a scale-free error measure as well, it can be used to compare performance across different data sets.

Examples

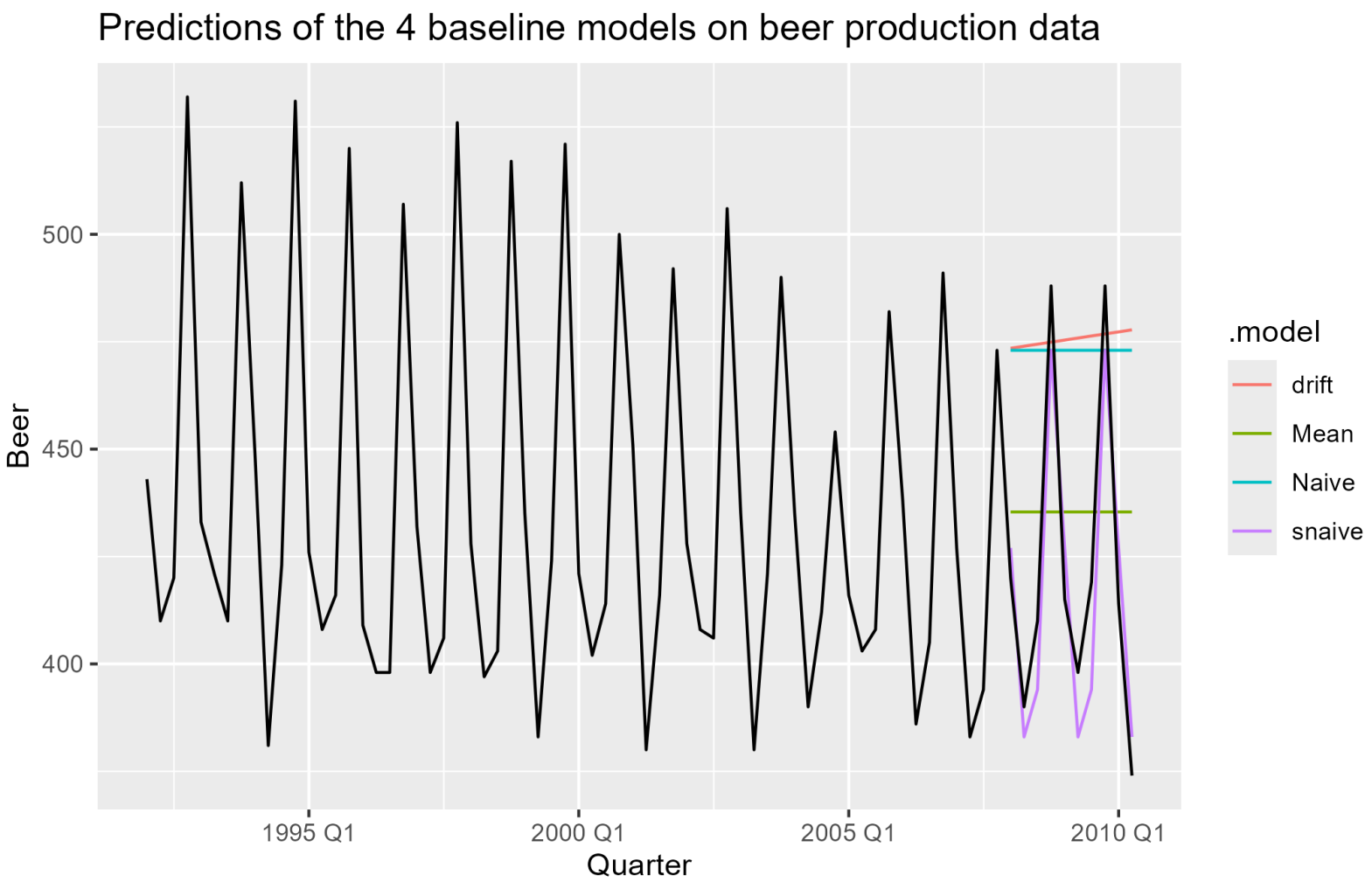
- We split part of the beer production data in two subsets: our training set will contain observations from 1992 to 2007, while the test set will contain observations from 2008 to the end (2010 Q2) - a total of 10 observation points.
- On the training data we will fit four simple models: the mean model, the naive model, the seasonal naive model and the drift model and get prediction for the next 10 observations
- We will plot them (original data - both for train and test period) and the four predictions to see how they fit
- We will evaluate the performance of these three models using the test data. For this we use the function *accuracy* from R, which displays the value of several error measures.

```r
allData <- aus_beer |>
  filter(year(Quarter) >= 1992)
trainData <- allData |>
  filter(year(Quarter) <= 2007)
testData <- allData |>
  filter(year(Quarter) > 2007)

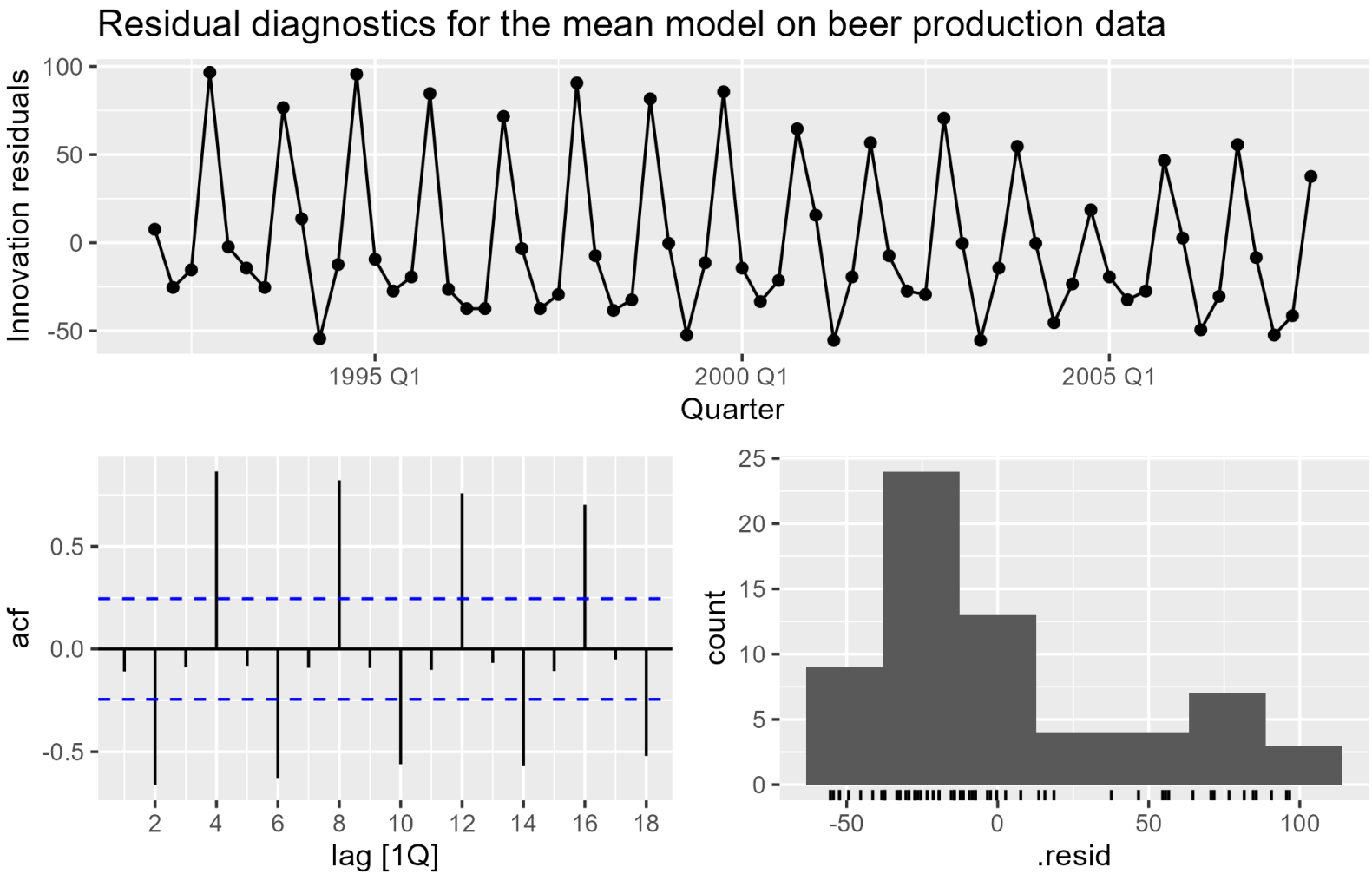
beer_models <- trainData |>
  model(Mean = MEAN(Beer),
        Naive = NAIVE(Beer),
        snaive = SNAIVE(Beer),
        drift = RW(Beer ~drift()))

beer_forecast <- beer_models |> forecast(h = 10)

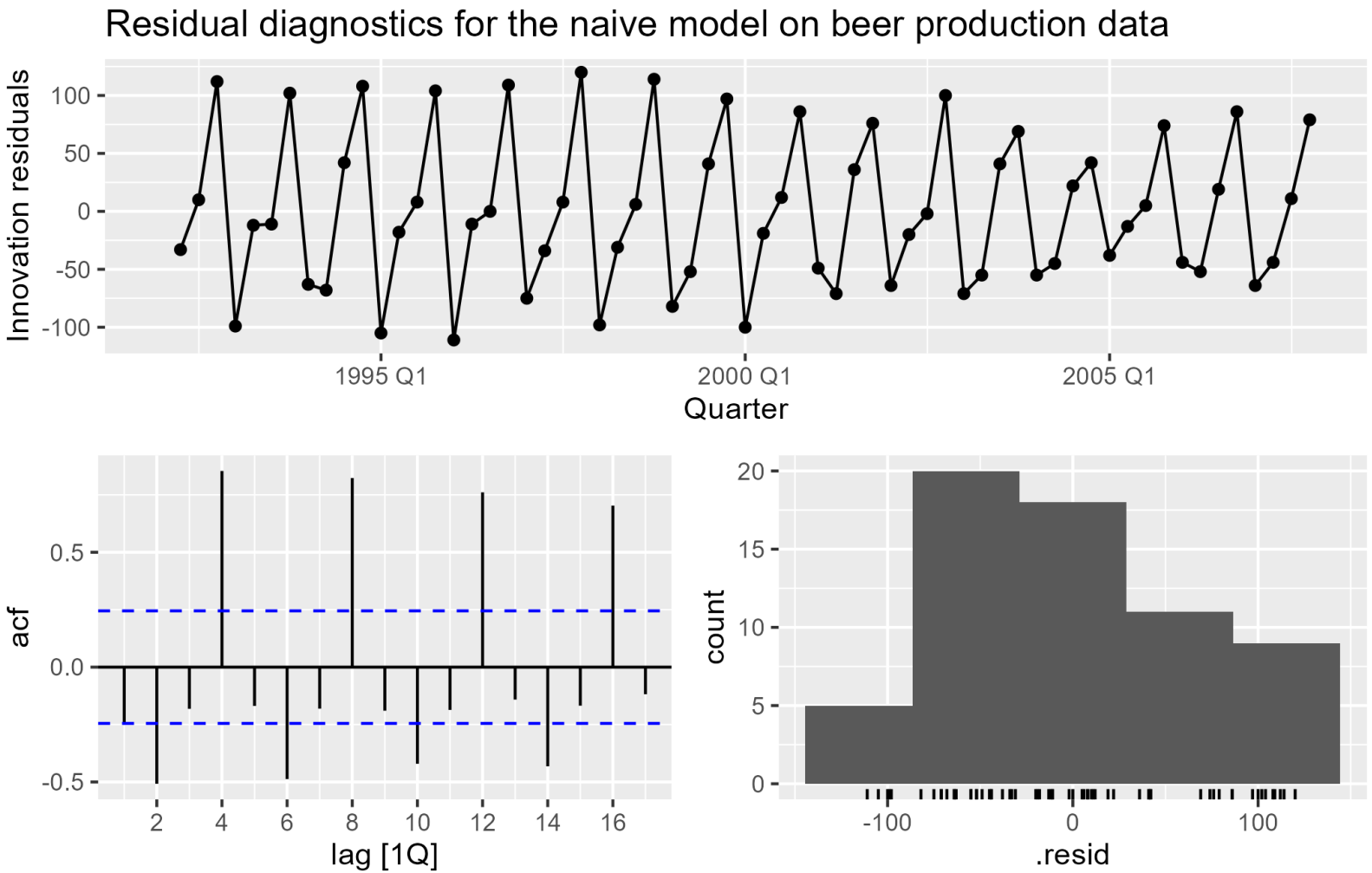
beer_forecast |>
  autoplot(allData, level = NULL) +
  guides(colous = guide_legend(title= "Forecast")) +
  labs(title = "Predictions of the 4 baseline models on beer production data")
```


Predictions of the 4 baseline models on beer production data
Beer
500
450
400
1995 Q1
2000 Q1
2005 Q1
2010 Q1
Quarter
.model
drift
Mean
Naive
snaive

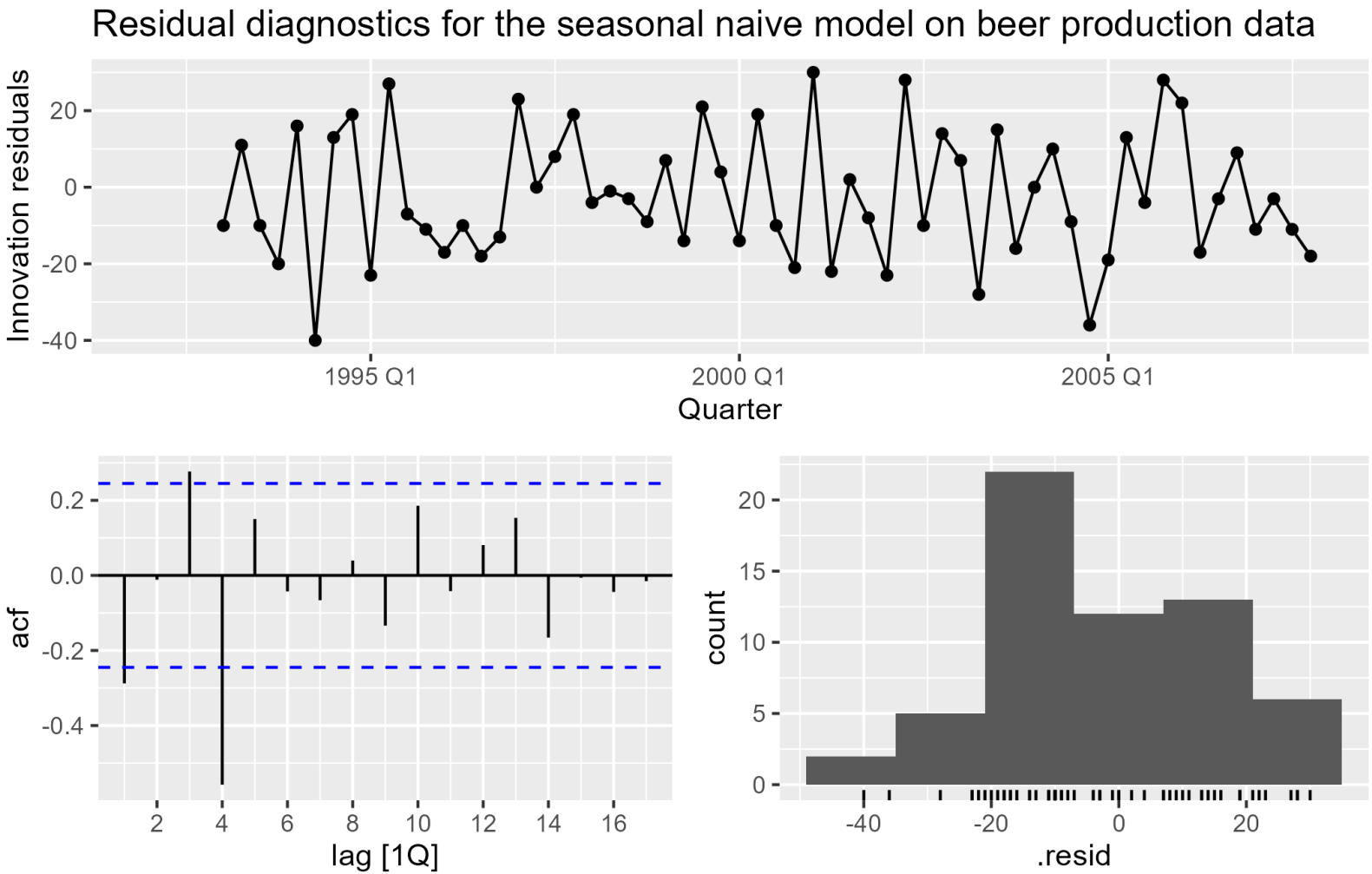
```
beer_models[1] |> gg_tsresiduals() +
  labs(title = "Residual diagnostics for the mean model on beer production data"
      ) #mean
```


Residual diagnostics for the mean model on beer production data
Innovation residuals
100
50
0
-50
1995 Q1
2000 Q1
2005 Q1
Quarter
acf
0.5
0.0
-0.5
2
4
6
8
10
12
14
16
18
lag [1Q]
count
25
20
15
10
5
0
-50
0
50
100
.resid

```
beer_models[2] |> gg_tsresiduals() +
  labs(title = "Residual diagnostics for the naive model on beer production data
     ") #naive
```


Residual diagnostics for the naive model on beer production data
Innovation residuals
100
50
0
-50
-100
1995 Q1
2000 Q1
2005 Q1
Quarter
acf
0.5
0.0
-0.5
2
4
6
8
10
12
14
16
lag [1Q]
count
20
15
10
5
0
-100
0
100
.resid

```
beer_models[3] |> gg_tsresiduals() +
  labs(title = "Residual diagnostics for the seasonal naive model on beer
      production data") # seasonal naive
```


Residual diagnostics for the seasonal naive model on beer production data
Innovation residuals
20
0
-20
-40
1995 Q1
2000 Q1
2005 Q1
Quarter
acf
0.2
0.0
-0.2
-0.4
2
4
6
8
10
12
14
16
lag [1Q]
count
20
15
10
5
0
-40
-20
0
20
.resid

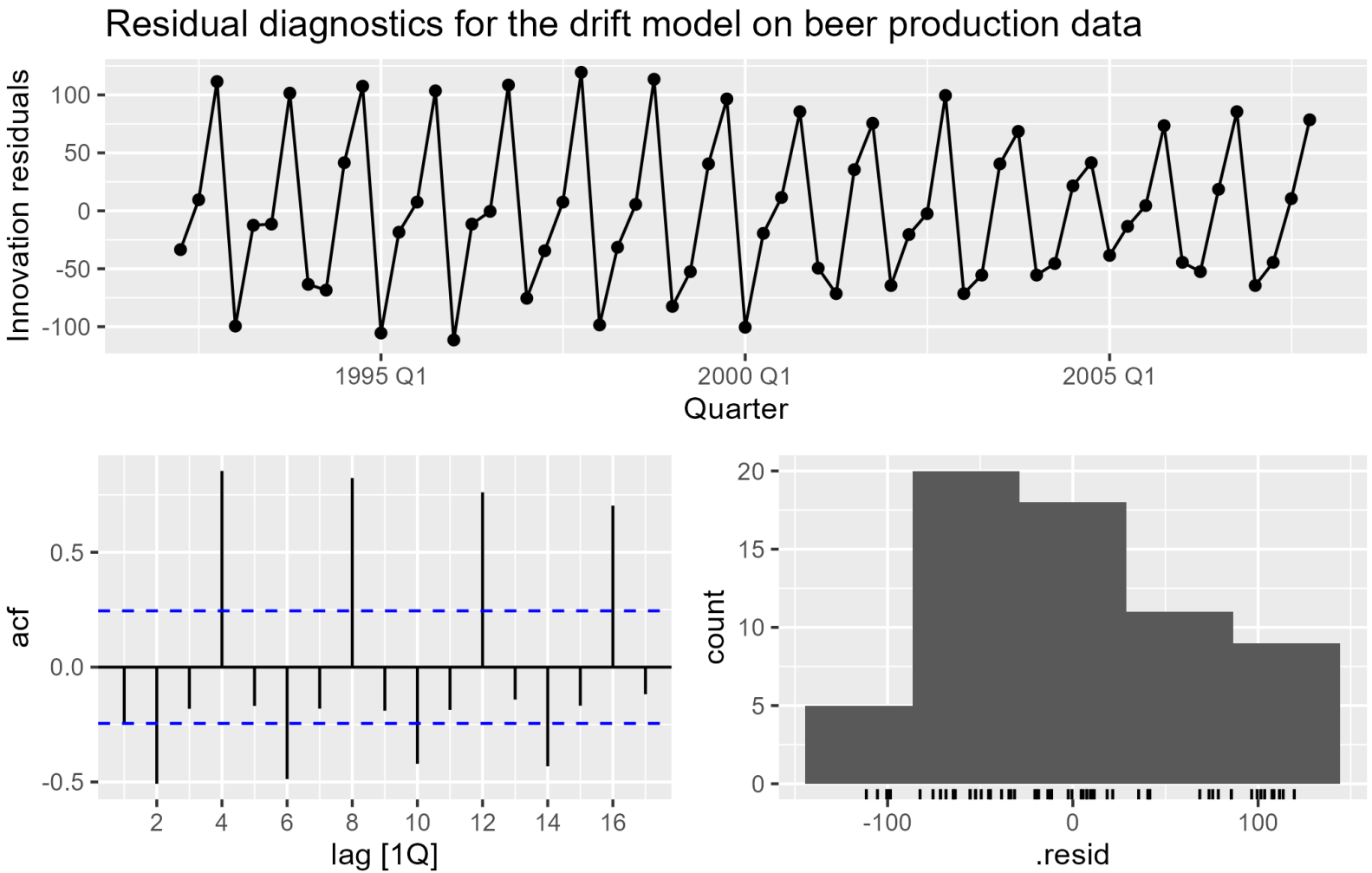
```
beer_models[3] |>  augment() |>
  features(.innov, box_pierce, lag = 8)
```

```
  .model bp_stat bp_pvalue
  <chr>    <dbl>     <dbl>
1 snaive    30.0  0.000208
```

```
beer_models[3] |>  augment() |>
  features(.innov, ljung_box, lag = 8)
```

```
  .model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 snaive    32.9 0.0000630
```

```
beer_models[4] |> gg_tsresiduals() +
  labs(title = "Residual diagnostics for the drift model on beer production data
     ")# drift
```


Residual diagnostics for the drift model on beer production data
Innovation residuals
100
50
0
-50
-100
1995 Q1
2000 Q1
2005 Q1
Quarter
acf
0.5
0.0
-0.5
2
4
6
8
10
12
14
16
lag [1Q]
count
20
15
10
5
0
-100
0
100
.resid

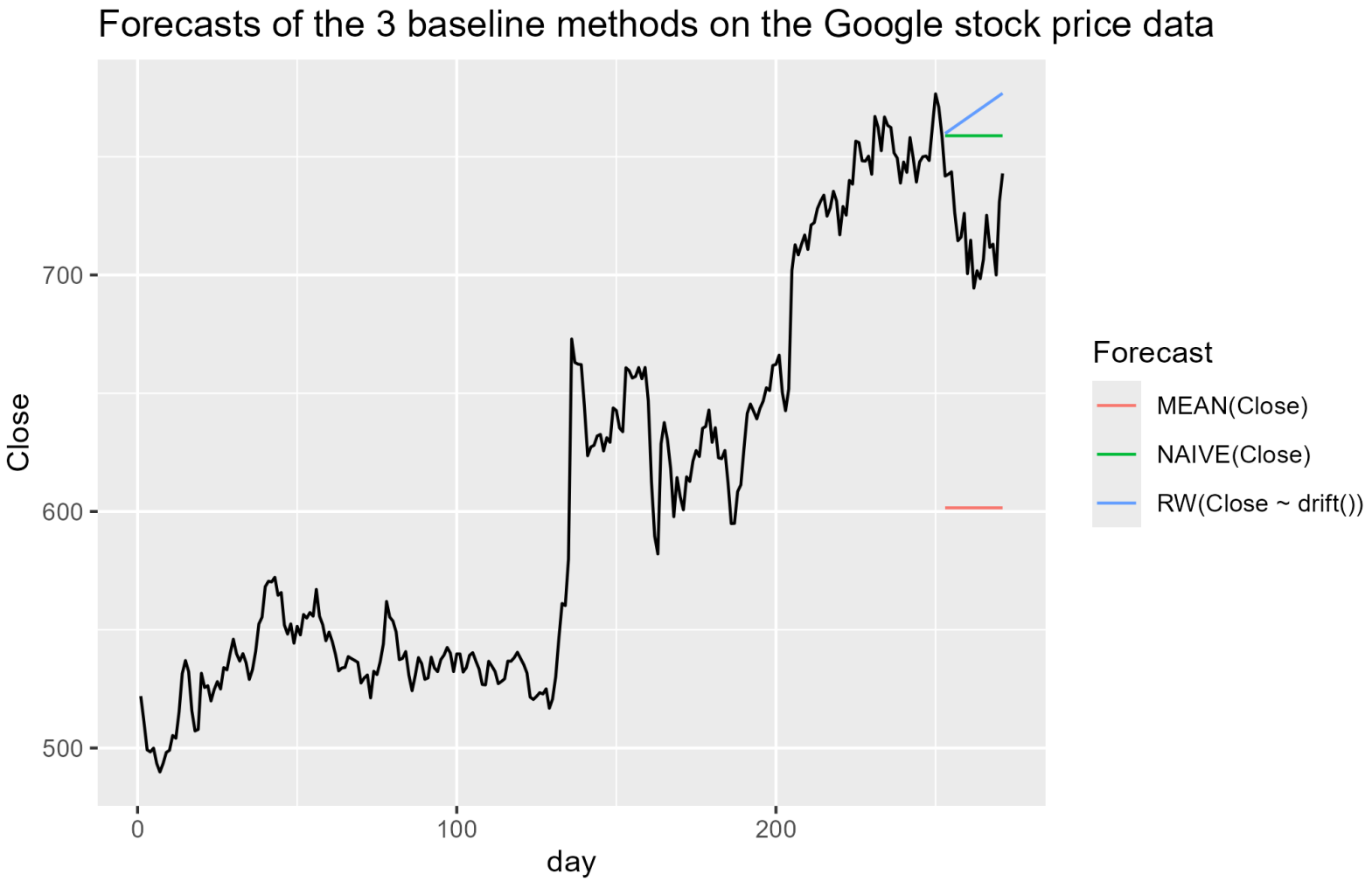
```
accuracy(beer_forecast, testData)
```

```
   .model .type    ME  RMSE   MAE    MPE  MAPE  MASE RMSSE    ACF1
  <chr>  <chr> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl>   <dbl>
1 Mean   Test  -13.8  38.4  34.8  -3.97  8.28   NaN   NaN -0.0691
2 Naive  Test  -51.4  62.7  57.4 -13.0  14.2    NaN   NaN -0.0691
3 drift  Test  -54.0  64.9  58.9 -13.6  14.6    NaN   NaN -0.0741
4 snaive Test    5.2  14.3  13.4   1.15  3.17   NaN   NaN  0.132
```

- According to all values, the seasonal naive model seems to be the best. However, residual diagnostics show that there still is autocorrelation in the residuals, so the model did not capture all patterns from the data.
- Let's consider a non-seasonal example as well, the Google stock closing prices from 2015 and 2016 (discussed already).

```
googleTrain |> model(MEAN(Close),
                     NAIVE(Close),
                     RW(Close ~drift())) -> googleModels

googleForecast <- googleModels |>
  forecast(googleTest)
googleForecast |>
  autoplot(bind_rows(googleTrain, googleTest), level = NULL) +
  guides(colour = guide_legend(title = "Forecast")) +
  labs(title= "Forecasts of the 3 baseline methods on the Google stock price
      data")
```


Forecasts of the 3 baseline methods on the Google stock price data
Close
700
600
500
0
100
200
day
Forecast
MEAN(Close)
NAIVE(Close)
RW(Close ~ drift())

```
accuracy(googleForecast, googleTest)
```

```
  .model              .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE  ACF1
  <chr>               <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 MEAN(Close)         Test  117.  118.  117.  16.2  16.2    NaN   NaN 0.496
2 NAIVE(Close)        Test  -40.4  43.4  40.4 -5.67  5.67   NaN   NaN 0.496
3 RW(Close ~ drift()) Test  -49.8  53.1  49.8 -6.99  6.99   NaN   NaN 0.604
```

- According to the measures the Naive model is the best, but on the plot we can see that all of them are quite bad.

Time series cross-validation

- Instead of doing a simple train-test split, in general, cross-validation is considered a better evaluation method.
- In case of cross validation, we have several training sets of increasing size and the test set is always the next observation (also called *evaluation on a rolling forecasting origin*). For example, for a time series with 20 observations, we could have the following cross validation-scenario:
  - Use the first 5 observations for training and observation 6 for testing (the rest is unused)
  - Use the first 6 observations for training and observation 7 for testing
  - Use the first 7 observations for training and observation 8 for testing
  - ...
  - Use the first 19 observations for training and observation 20 for testing
- For each test case we will have an error (or accuracy) measure and their average is the performance of the method.

- Cross-validation can be performed for multi-step forecasts as well. For example, you can use the first X observations for training and forecast instance $X + 4$ (but only one instance).
- **Obs.** It needs to be carefully analyzed how many observations are needed initially (on one hand you would like to do as many rounds as possible, on the other hand, you cannot train a good model on too few training instances).
- Cross-validation is implemented in the *strech_tsibble* function, which creates many training sets. You need to pass it the initial training set length, *.init* (i.e. the minimum number of observations to use for training) and the increase in training set size, *.step*.

```
google_2015_tr <- googleTrain |>
  stretch_tsibble(.init = 3, .step = 1)
google_2015_tr |> view()

google_2015_tr |>
  model(RW(Close ~drift())) |>
  forecast(h = 1) |>
  accuracy(googleTrain)
```

```
  .model               .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE   ACF1
  <chr>                <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 RW(Close ~ drift()) Test  0.726  11.3  7.26 0.112  1.19  1.02  1.01 0.0985
```